
清雁科技（北京）有限公司实习技术工作总结

面向金融知识库的 RAG + 思维图多步推理问答系统 研发实践报告

摘要

随着数据资产产业化的快速发展，金融行业对高精度知识问答的需求日益增长，但传统大模型在专业领域存在术语理解不足、推理链条断裂等问题。本研究提出一种基于**思维图多步推理**的大模型认知强化方法，融合**检索增强生成(RAG)**技术与**Agent 思维图推理**框架，提高企业级智能知识库问答场景精度。

研究首先针对金融领域多源异构数据，设计三级切片策略与动态上下文继承机制，通过 **FAISS** 向量数据库实现高效语义检索。在查询阶段，采用“问题分解 — 混合检索 — 重排序 — 自我验证”流程，结合 **DeepSeek-R1** 大模型生成结构化回答，并通过 **MCP** 协议优化工具集成与多智能体协作。实验结果表明，该方案在金融数据集上实现 87.05% 的问答准确率，较基线模型提升 18.09%，且在长链推理、多模态适配等方面表现出显著优势。消融实验验证了 PDF 结构化转换、重排序模块及自我验证机制的有效性，参数优化确定 **text-embedding-v3** 嵌入模型与 8 片段重排序策略为最优配置。

本研究为垂直领域大模型应用提供了可复用的技术框架，未来计划引入量子机器学习与多模态解析技术，进一步提升系统泛化能力与推理效率。

关键词：大模型认知强化；思维图多步推理；RAG 技术；MCP 协议

目录

第 1 章 绪论	1
1.1. 研究背景与意义	1
1.1.1. 市场需求背景	1
1.1.2. 研究的意义	1
1.2. 国内外研究现状与分析	2
1.2.1. 国内外研究现状	2
1.2.2. 现有研究不足分析	5
1.3. 研究内容与目标	6
1.3.1. 研究目标	6
1.3.2. 研究内容	7
1.4. 论文组织结构	7
第 2 章 方法	8
2.1. 基础路线	8
2.2. 向量知识库构建	8
2.2.1. 多元异构文档读取	8
2.2.2. 长文档切片策略	9
2.2.3. 文档片段依次嵌入向量数据库	10
2.3. 查询	11
2.3.1. 检索召回相关的片段	12
2.3.2. 重排序筛选核心片段	13
2.3.3. 基于检索片段组织回答	13
2.3.4. 自我验证	13
2.4. 本章小结	14
第 3 章 实验结果与分析	15
3.1. 实验配置	15
3.1.1. 采用的数据集	15
3.1.2. 评价指标	16
3.1.3. 基准对照模型	16
3.1.4. 实现细节	17
3.1.5. 参数优化策略和最终设置	19
3.2. 基础实验测试	20
3.2.1. 基于 Ollama 部署模型进行测评	20

3.2.2. 基于服务商大语言模型测评	22
3.2.3. 最终方案	23
3.3. 详细实验	23
3.3.1. 消融实验	23
3.3.2. 嵌入模型选择	25
3.3.3. 检重排序检索片段数量参数确定	26
3.3.4. 生成模型选择	27
3.3.5. 模型效率测试	28
3.3.6. 模型普适性测试	29
3.4. 应用示例	30
3.4.1. 模型上下文协议	30
3.4.2. 制作界面展示 MCP 多客户端管理平台 demo	31
3.5. 本章小结	32
第 4 章 总结与展望	33
4.1. 主要内容及结论	33
4.2. 主要创新点和贡献	33
4.3. 未来工作展望	34
参考文献	36

第 1 章 绪论

1.1. 研究背景与意义

1.1.1. 市场需求背景

关于数据资产产业化发展状况，依据 IDC 所发布的《全球数据圈报告 2025》中的相关内容可知，全球每年新增加的数据量达到了 235ZB 之多，企业当中结构化数据所占的比例为 32.1%，中国的数据产量呈现出复合增长率达到 30.4% 的态势，金融机构的数据存储总量是 15.6ZB。根据国家工业信息安全发展研究中心所披露的数据说明，在 2024 年的时候我国的数据要素市场规模突破了 2000 亿元，其中金融、医疗、制造这三个行业的数据资产化率分别是 41.2%、28.7%、19.3%。

金融行业对于数据处理有着特殊的需求，根据银保监会所公布的数据可看出，在 2024 年的时候，银行业平均每天处理的交易数据数量为 4.3 亿条，其中包含敏感字段的数据所占的比例达到了 67%，依据 SWIFT 协会的统计结果发现，全球范围内的金融机构因为数据识别出现错误，平均每年造成的损失高达 47 亿美元。

垂直领域技术适配的当前状况，依据 Gartner 报告所呈现的内容，在 2024 年的时候，企业知识库问答系统行业的定制需求出现了大幅度的增长，增长幅度高达 240%，其中金融业在该系统中的部署占比达到了 34%，根据清华大学 AI 研究院所进行的测试结果显示，当引入 RAG 技术之后，金融合同条款检索的准确率得到了较大提升，从原本的 76% 提升至 94%。然而在专业术语推理方面，仍然存在一定的误差，误差率为 23%。

1.1.2. 研究的意义

本研究项目的落地背景处于金融行业，借助将 RAG 技术与 Agent 思维图推理技术相融合的方式，构建出高精度且可定制的企业知识库问答系统，而且本研究有着**较高的可迁移性**，对于那些依赖高精度知识问答的其他行业同样有着较高的适用性。促使 RAG 技术实现落地应用，当下的 RAG 系统在如金融、法律等专业领域当中，依旧存在着对于行业术语理解不够充分，以及检索结果相关性较低等一系列问题，此项研究借助优化检索策略以及提高语义匹配能力的方式，来提升 RAG 技术在金融知识领域的问答准确程度。**促进 Agent 思维图推理可在实际中得到应用**，当下已有的 Agent 系统在如风险评估、合规审查这类复杂的金融场景当中，其推理能力以及提升的空间，此项研究引入了思维图技术，借助 Agent 更为系统地去分析金融数据、剖析复杂问题，生成符合行业规范的回应。**帮助数据资产行业发展，推动传统行业数字化转型进程加速**，当下不少金融机构存在数据利用

率不高、知识沉淀存在险阻等状况，本研究中的企业知识库问答技术，可有效挖掘金融数据潜在价值，达成数据资产智能化管理，而且此项研究成果可为医疗、法律等别的行业给予可复用的技术方案，促使数据资产在更广阔领域实现价值转化，推动传统行业向智能化方向升级。

1.2. 国内外研究现状与分析

1.2.1. 国内外研究现状

近年来，学界围绕**大语言模型推理能力**提升展开多元探索，从方法综述、框架创新到工具整合与多模态推理，形成一系列突破性成果，为复杂任务解决提供了丰富思路。Yu^[1]等人在 2025 年对 Chain-of-X 方法进行了全面综述，分类分析了其在不同任务中的应用，展示了其在增强大语言模型推理能力和多模态交互等方面的潜力。Zheng Chu^[2]等人系统调查了链式思考推理研究，提出了精细分类法，探讨了前沿问题和未来方向，为未来研究提供指引。Maciej Besta^[3]等人提出了一个关于大型语言模型（LLM）推理方案的通用蓝图和分类法。他们通过深入分析推理流程，明确了关键概念，并比较了不同设计选择对性能和成本的影响，为未来 LLM 推理技术的发展奠定了基础。Bowen Jin^[4]等人提出了 Graph Chain-of-Thought（GRAPH-COT）框架，通过迭代推理增强大语言模型（LLMs）对图结构知识的利用，有效解决了 LLMs 在知识密集型任务中的幻觉问题，显著提升了模型在图推理任务中的表现。武汉大学李正^[5]等人提出了 ReX-GoT 框架，通过逐步排除无关选项并结合图推理，有效解决了对话常识多选问答任务中的选项饱和和线索复杂问题，显著提升了模型性能。Du Phan^[6]等人提出一种结合隐变量推理的链式思考（CoT）微调策略，通过 MCMC-EM 算法优化模型生成正确答案的边际对数似然，无需详细标注推理过程，显著提升了 LLM 在 GSM8K 和 BIG-Bench Hard 任务上的准确率。Ma^[7]等人提出了一种名为 SCIAGENT 的工具增强型语言模型，用于科学推理任务。他们构建了一个包含 3 万多个样本和 6 千多个工具的训练语料库 MATHFUNC，并开发了 SCITOOLBENCH 基准测试，覆盖数学、物理、化学等五个科学领域。实验表明，SCIAGENT 在科学推理任务中显著优于其他同类模型，提升了工具使用能力和推理准确性。Yixuan Weng^[8]等人提出了一种利用大型语言模型（LLMs）进行自我验证的方法，通过在推理任务中增加反向验证步骤，显著提升了模型在算术、常识和逻辑推理数据集上的性能，为提升 LLMs 的推理能力提供了新思路。张丹^[9]等人提出了一种名为 ReST-MCTS* 的强化自训练方法，通过结合过程奖励引导的蒙特卡洛树搜索（MCTS*）来收集高质量推理轨迹及每步价值，用于训练策略和奖励模型，显著提升了大型语言模型在复杂推理任务上的性能。Carnegie Mellon University^[10]的研究团队构建了一个名为 WebArena 的高度逼真且可复现的网络环境，用于

开发和测试通过自然语言指令执行任务的自主智能体，填补了现有研究中简化环境与真实场景之间的差距。Xiangyan Liu^[11]等人提出了 M3 框架，针对多模态多步推理中的模型选择问题，通过建模子任务依赖关系，显著提升了多模态代理在复杂任务中的推理鲁棒性。Junnan Liu^[12]等人提出了 KNOWFORMER，这是一种基于 Transformer 架构的知识图谱推理方法，通过引入结构感知模块和高效的注意力计算机制，解决了传统路径方法中信息丢失和过压缩的问题，显著提升了知识图谱推理的性能。Andy Zhou^[13]等人提出了 Language Agent Tree Search (LATS)，将语言模型的推理、行动和规划能力融合，通过蒙特卡洛树搜索和环境反馈提升决策性能，实现了多领域任务的高效解决。Yue Wu^[14]等人提出了 SmartPlay，是一个用于评估大型语言模型 (LLMs) 作为智能代理的基准测试框架，包含 6 款游戏，挑战 LLMs 的推理、规划、空间认知等 9 种关键能力，为识别 LLMs 的能力缺口和推动智能代理研究提供了有效工具。北京大学、北京信息科技大学和北京师范大学的研究人员^[15]共同提出了一种基于大型语言模型 (LLM) 的自演化 AI 外交官代理 Richelieu，通过整合战略规划、社交推理和自我进化等能力，在复杂多智能体任务中展现出超越现有水平的外交能力。上述研究覆盖综述分析、框架设计、工具增强及多模态推理等维度，实为大语言模型的理论基础，为其在科学推理、复杂决策等场景的落地提供了实践路径。

近年检索增强生成 (RAG) 与大语言模型结合的研究蓬勃发展，涵盖技术综述、数据分布优化、架构创新及垂直领域应用，通过多方法融合持续提升模型在知识密集型任务中的性能与实用性。Yunfan Gao^[16]等人对检索增强型生成 (RAG) 技术进行了全面综述，分析了其在大型语言模型 (LLMs) 中的发展历程、核心技术和应用，并提出了当前挑战与未来发展方向，为相关研究提供了系统性参考。Nikhil Kandpal^[17]等人研究了大型语言模型在预训练数据中知识分布不均时的学习表现，发现模型对长尾知识的学习能力与预训练数据中相关文档数量密切相关，揭示了模型在稀有知识学习上的局限性。黄红伟等人^[18]提出了语义特征空间模型，优化了基于 RAG 架构的智能问答系统，通过改进文本向量化策略，显著提高了文本召回率，为智能问答领域提供了新的技术思路和方法。黄奕明^[19]等人提出了一种预回答与召回过滤 (PARF) 双阶段优化方法，通过结合领域知识图谱与提示工程技术引导大语言模型生成预回答，并利用 BERT 模型对召回文段进行相关性评分与过滤，显著提升了 RAG 问答系统在垂直领域的回答准确性和全面性。Luyu Gao^[20]等人提出了一种名为 HyDE 的零样本密集检索方法，通过利用指令遵循的语言模型生成假设文档并结合无监督对比编码器进行检索，无需相关性标签，显著提升了零样本检索性能。Zhuyun Dai^[21]等人提出了一种名为 PROMPTAGATOR 的方法，利用大型语言模型 (LLM) 作为少样本查询生成器，通过仅使用少量标注样本（如 8 个）生成大量合成数据来训练任务特定的检索器，显著提升了少样本密集检索的性能，超越了基于大规模数据集（如 MS MARCO）训练的模型。Keming Lu^[22]等人提出了 MR.COD 方法，用于跨文档关系抽取中的多跳证据检索，通过挖掘和排序证据路径，有效提升了跨文档关系抽取的性能，验证了高质量证据

检索对端到端关系抽取的重要性。Raviteja Anantha^[23]等人提出了 Context Tuning 方法，通过智能检索相关上下文信息，显著提升了检索增强生成（RAG）中工具检索和计划生成的效果，减少了幻觉现象。Shicheng Xu^[24]等人提出了一个名为 Search-in-the-Chain (SearChain) 的新框架，通过在大型语言模型（LLM）和信息检索（IR）之间进行交互，解决了复杂知识密集型任务中多步推理的准确性、可信度和可追溯性问题，显著提升了任务性能。Trivedi^[25]等人提出了一种名为 IRCot 的新方法，通过将检索与链式思考（CoT）推理步骤交替进行，显著提高了多步问答任务中的检索效果和下游问答性能，减少了模型幻觉现象，提升了推理的准确性。Fangyuan Xu^[26]等人提出了一种名为 RECOMP 的方法，通过压缩检索到的文档为文本摘要，再将其前置到语言模型输入中，显著降低了计算成本，同时在语言建模和开放域问答任务中保持了性能。吴斌峰^[27]等人基于海上平台专业知识库，利用检索增强生成和参数高效微调方法，构建了针对 FPSO 生产流程的智能问答系统，有效提升了模型在专业领域的问答准确性和实用性。杨诗卓^[28]等人设计了一种基于 Qwen2.5 大语言模型的医疗问答系统优化框架，融合提示词工程、RAG 检索增强和 LoRA 微调技术，构建医学领域专用问答数据集，显著提升了医疗问答的准确性、术语规范性和推理逻辑性。江新科^[29]等人提出了一种基于假设知识图增强（HyKGE）的医学大语言模型框架，通过整合知识图谱提升模型在医学领域的准确性和可解释性，显著提高了复杂场景下的知识检索与回答质量。Yu Wang^[30]等人提出了一种基于知识图谱提示（KGP）的方法，用于多文档问答（MD-QA），通过构建知识图谱并设计图遍历模块，为大型语言模型（LLM）提供合适的上下文，显著提升了多文档问答的性能。Huiqiang Jiang^[31]等人提出了 LongLLMLingua，通过提示压缩技术解决长文本场景下大语言模型计算成本高、性能下降和位置偏差的问题，显著提升了模型性能并降低了成本和延迟。Tom Kwiatkowski^[32]等人构建了 Natural Questions 数据集，包含真实用户查询和维基百科页面标注的长答案与短答案，为开放域问答研究提供了大规模高质量训练数据，推动了自然语言理解的发展。Angela Fan^[33]等人构建了 ELI5 数据集，包含 270K 篇 Reddit 论坛的问答，用于长文本问答任务，推动了复杂问题多句回答的模型研究。Charles Packer^[34]等人提出 MemGPT，借鉴操作系统虚拟内存管理技术，通过分层存储和函数调用扩展大型语言模型（LLM）的上下文窗口，使其在长文档分析和多轮对话中突破固定上下文限制，显著提升性能。Rulin Shao 等人^[35]构建了 1.4 万亿 token 的 MASSIVETS 数据集，研究了检索式语言模型在推理时数据量对性能的影响，发现大数据集可显著提升模型表现，尤其在知识密集型任务上超越了更大规模的纯语言模型。复旦大学的王新涛^[36]等人提出了一种名为 KnowledGPT 的框架，通过程序化思维提示，将大型语言模型与知识库连接，增强了模型的知识检索和存储能力，有效提升了回答世界知识相关问题的准确性和范围。Akari Asai^[37]等人提出了一种名为 Self-RAG 的框架，通过自反思机制实现按需检索和生成文本，显著提升了大型语言模型在多种任务中的事实性和生成质量，特别是在事实验证和长文本生成任务中表现突出。Avik Dutta^[38]等人提出了一种

名为 Retrieval Augmented Retrieval (RAR) 的两步检索方法,用于低资源编程语言的代码生成。该方法通过结合示例和文档信息,显著提升了代码生成的准确性和相关性,解决了现有方法在低资源语言中生成效果不佳的问题。Boxin Wang^[39]等人提出了 InstructRetro,在检索增强预训练的基础上进行指令微调,通过继续预训练 43B GPT 模型并检索 1.2 万亿 tokens,得到的 Retro 48B 模型在零样本任务上显著优于 GPT,为获取更好的 GPT 解码器提供了新方向。张哲斌^[40]等人提出了一种名为 IAG 的诱导增强生成框架,用于回答隐式推理问题,通过结合检索文档和从大型语言模型中诱导的知识,显著提升了开放域问答任务的性能。Yubing Ren^[41]等人提出了一种基于混合检索增强的文档级事件参数抽取方法,通过设计不同的检索策略并引入伪示例,有效提升了模型的类比能力和抽取性能。Haeun Yu^[42]等人提出了一种统一框架,对比了语言模型中实例归因 (IA) 和神经元归因 (NA) 方法,揭示了它们在解释模型参数知识方面的差异和互补性,为深入理解语言模型的内部机制提供了新视角。浙江大学团队^[43]开发了 TableGPT 框架,将表格、自然语言和命令统一到一个 GPT 模型中,通过全局表格表示和指令链实现高效的数据分析与可视化,提升了表格数据处理的便捷性和适应性。邵志宏^[44]等人提出了一种名为 ITER-RETGEN 的方法,通过迭代检索与生成协同的方式增强检索增强型大语言模型,有效提升了复杂信息需求任务的性能,同时减少了检索和生成的开销。Wenhao Yu^[45]等人提出了一种名为 CHAIN-OF-NOTE (CON) 的新方法,通过为检索到的文档生成序列化笔记来增强检索增强型语言模型 (RALM) 的鲁棒性,有效提升了模型在面对噪声文档和未知场景时的表现。上述成果从理论分析到技术落地,深化了 RAG 与 LLMs 在检索机制、数据利用等方面的协同,为医疗、工业等专业领域提供了定制化解决方案。

1.2.2. 现有研究不足分析

多源异构数据整合面临难题,企业数据囊括结构化与非结构化数据,在对 Excel、PDF 表格这类半结构化数据进行向量化处理时,原有层次结构信息会出现 35% 的丢失情况,不同业务系统的数据标准存在差异,字段映射错误会致使知识关联丢失,对后续检索准确率产生影响。增强型 RAG 实现阶,在**专业术语查询**过程中,传统的向量检索方式所呈现出的召回率处于 82%至 88%这一区间范围内,这也就意味着存在 12%至 18%的可能性会漏检关键文档,而在长文档检索时,会出现信息被截断的情况,平均而言,会丢失 22.7%的关键内容。**动态知识**在有效更新方面存在险阻,增量索引构建所需要的时间以及数据量呈现出非线性增长的态势,新旧知识冲突检测的准确率仅为 89.3%,这使得知识库内部矛盾率会随着时间不断增加。**复杂问题的分解**面临险阻,难以达到有效分解的效果,对于 5 步以上的问题进行自动拆解时,其准确率仅为 76.8%,同时关键子任务的遗漏率为 14.3%,另外推理效率方面存在瓶颈。

1.3. 研究内容与目标

1.3.1. 研究目标

首先对企业专有数据开展预处理，将如文档、数据库等多源异构数据给予标准化清洗，使清洗准确率目标高于 98%，其次开发增量式更新机制，把新数据整合延迟控制在 2 小时以内，使知识库覆盖率达到 95%以上，最后完成实体关系自动抽取算法开发，覆盖企业核心业务概念达 90%以上。

提高型 RAG 技术的开发包含多个方面，其一要达成混合检索系统的构建，将稠密向量检索技术与关键词检索技术进行整合，其二需运用基于业务规则的重排序模型，其三要对上下文感知方面加以优化，其四构建多粒度检索体系，使其可在从概念级到细节级的范围内都实现精准定位。思维图推理框架的构建覆盖多个方面：其一，要达成问题分解与规划模块，此模块可支持自动拆解超过 5 步的复杂问题，其二，需支持对推理过程进行可视化呈现，同时还要支持生成推理报告。

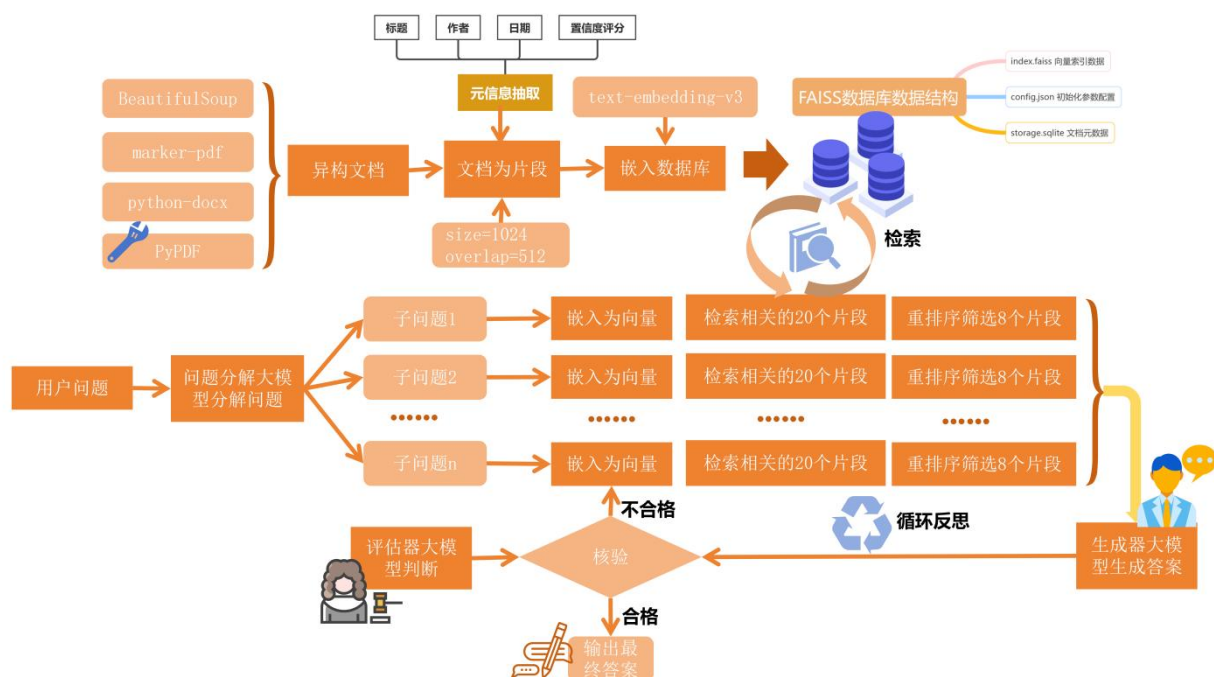


图 1.1 全局思维导图

开发高效企业智能体，首先要开展轻量化模型部署工作，其次是开发微服务架构，以此来支持模块化部署以及弹性扩展，最后是基于 MCP 协议达成 API 热插拔管理。系统集成与验证工作包含两项任务，一是针对端到端性能展开优化，运用多级缓存策略，使得高频查询的响应时间可控制在小于 200 毫秒的范围内，二是在金融领域实施测试，专业术语

的理解准确率可达到 89.7%以上。

1.3.2. 研究内容

如图 1.1 所示，本次研究主要分建库和查询两个阶段，**在建库阶段**，通过多种不同的文档解析工具，自动适配文档类型并进行检测。考虑到大模型输入输出上下文最大长度限制，这里将长文本进行切片保存，同时将整个文档的元信息提取出来，以增强同一个文档不同片段之间的相关性。最后采用 `text-embedding-v3` 模型将文档片段嵌入到 FAISS 向量数据库中。**在查询阶段**，用户输入的问题会从多个维度被分解，然后被嵌入为向量，到建立好的 FAISS 数据库中检索最相关的信息片段，然后经历重排序算法进行二次筛选，提高信息命中率。最后将所有片段给到生成器大模型做回答生成，将生成的结果给到评估器大模型，如果评估器大模型认定生成合格，则顺利返回生成的答案，如果不合格，则进入反思循环，结合历史记录重新进行问题的检索和生成。

1.4. 论文组织结构

绪论着重关注研究背景与意义，经过对国内外研究现状加以剖析，确定本研究核心在于将 RAG 与 Agent 思维图推理技术相融合，并设计了详细的实现框架。**在分析任务章节中**，对核心任务和挑战给予拆解，知识库要达成多源数据的标准化清洗，要求准确率达到 98%以上，实现增量更新，延迟需控制在 2 小时以内，并且要完成实体关系抽取，覆盖 90%的核心概念，RAG 技术需要对混合检索、重排序以及上下文感知能力进行优化，思维图框架要可支持 5 步及以上复杂问题的拆解以及推理可视化。

方法部分详细阐述技术路线以及核心模块，采用“文档解析→三级切片→向量建库→检索生成→自我验证”的流程，借助 PyPDF2 和 Markdown 来解析多种格式的文档，按照 `chunk_size = 1024`、`overlap = 512` 进行切片，同时保留标题语义坐标，运用 FAISS 构建 IVF_FLAT 索引库。在查询阶段，依靠 FAISS 倒排索引召回候选片段，经过重排序模型优化后，由 DeepSeek - R1 生成答案，并借助 BERTScore 与 NLI 模型验证逻辑一致性，形成从数据处理到推理验证的完整链路。

在实验结果与分析这一章节中，所依据的数据集包含企业财报以及 GPT - 4 提高数据等内容，依靠运用自动化评分机制来对系统性能进行验证，基础实验说明服务商模型相较于本地模型有优势，消融实验说明 PDF 转 Markdown、重排序、检验机制等模块发挥了有效作用，最终使得准确率达到 87.05%。参数优化明确了 `text - embedding - v3` 作为最优嵌入模型，重排序片段数 8 作为最佳配置，并且对模型在新数据以及政策文件中的普适性进行了验证。**在总结与展望这一章节之中**对研究成果给予了总结。

第2章 方法

2.1. 基础路线

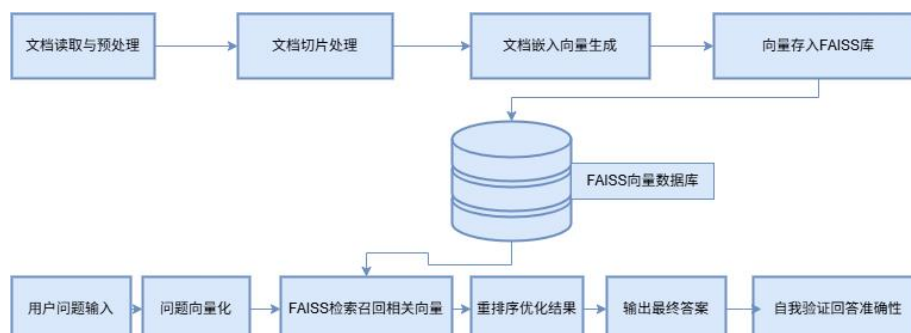


图 3.1：基础工作路线

如图 3.1 所示，首先对预处理文档给予读取，经过切片操作后生成嵌入向量，将其存入 FAISS 库，当用户提出问题时，先把问题进行向量化处理，之后借助 FAISS 进行检索召回，接着对结果实施重排序优化，再结合提示词与证据来生成回答，随后凭借自我验证以保证回答的准确性，最终输出答案。

2.2. 向量知识库构建

对于数据库部分，此处选用 faiss 来构建数据库，FAISS 即 Facebook AI Similarity Search，是由 Facebook AI Research 所开发的开源库，主要用于实现高效的大规模相似性搜索以及聚类操作，它支持多种索引方法，像 Flat、IVF、HNSW 等，同时也支持 L2 距离、余弦相似度等度量方式。该库拥有高效处理数十亿向量的能力，有很强的可扩展性，可动态地进行向量的增删改操作，其参数灵活，并且开源特性使得它易于定制，它适用于对速度有较高要求的大规模向量检索场景，比如 RAG 系统的知识库构建，可契合高效检索的需求。

2.2.1. 多元异构文档读取

借助多格式适配技术，比如 PyPDF2、python-docx 等，可从 PDF、DOCX、HTML 等不同类型的异构文档里自动提取文本内容以及元数据，如作者、日期、标题等信息，以此解决格式碎片化的问题，为后续的切片处理、向量嵌入以及知识库构建提供有结构化且完整的基础数据，保障文档信息可以得到有效利用，并且提升检索的准确性。

文档格式适配层会借助文件扩展名识别机制来自动调度相应的解析器，对于 PDF、DOCX 这类非结构化文档，会采用基于 PyPDF2 和 python-docx 的文本抽取算法，对于 HTML 文档，则会实施 DOM 树解析策略，运用 BeautifulSoup 库来实现结构化文本提取。而对于半结构化数据源，比如 Excel 或者复杂 PDF，会采用 marker 工具进行格式转换预处理，凭借生成标准 Markdown 格式来有效保留原始表格结构以及层级关系等语义信息。

元数据智能抽取所涉及的内容覆盖了多个方面，其中包括类型识别、标题提取、作者检测以及日期解析等四个维度的元数据。结构化存储引擎运用 Neo4j 图数据库达成文档实体关系建模，其中每个文档节点依靠 URL 给予唯一标识，在进行持久化存储之际会自动关联原始文件路径、处理时间戳等系统元数据。异常处理机制可达成三级容错体系，其中文件格式预检模块会对文件魔术字节展开验证工作，在解析过程中，会采用重试策略，其最大尝试次数设定为 3 次，对于 LLM 输出，会实施强制格式校验，当遇到缺失字段时，会自动填充默认值，同时记录异常日志。

2.2.2. 长文档切片策略

把长文档划分成适宜模型处理的语义单元，如 1024 字符的片段，运用三级切片策略达成上下文语义继承，这种策略包含基础切片、语义提高以及动态适配，以此避免信息被截断，控制计算粒度来适配模型最大上下文窗口，如 BERT 的上下文窗口有 4096 的长度限制，为后续向量嵌入和高效检索提供结构清晰且语义完整的基础单元。

1. 文档切片模块技术架构

借助三级切片策略达成文档的智能化分割。在基础切片层，依据递归字符分割算法，借助预先设定的分隔符集来开展均值切片操作，以此保证基础语义单元的完整性。在语义提高层，剖析 Markdown 标题结构，构建四级标题索引体系，依靠标题路径绑定达成语义上下文继承。在动态适配层，按照文档类型自动对切片策略给予调整。

构建一个双层元数据注入体系，对于基础元数据部分继承了文档类型、标题、作者以及日期等核心属性，而扩展元数据部分，则是注入了置信度评分、用户定义分类等动态属性，采用前缀标识法，如“【片段基础信息】”来达成结构化元数据融合，以此提高检索阶段的特征关联度。采用 FAISS 向量库，其混合索引策略能平衡精确检索与近似检索，有增量更新机制，可支持动态文档追加，版本控制方面，凭借 project_path/doc_id 双层级来隔离不同项目的数据。

2. 技术创新点

动态上下文继承机制。借助标题路径追溯算法来生成语义坐标描述，以此让每个文本片段可带有完整的结构上下文信息，经过实验发现，这样的设计使得检索准确率有了一定程度的提升，提升幅度为 5.2%。**混合存储策略。**运用 FAISS 的 IVF_FLAT 索引类型，在数据规模达到百万级别的情况下，可达成检索延迟小于 50ms 的效果，还可维持 95% 以上的召回率。格式兼容设计。借助正则表达式提高引擎达成 Markdown 与自然文本的双模解析。

2.2.3. 文档片段依次嵌入向量数据库

1. 理论部分

文档嵌入是把文本映射至低维连续向量空间的一种方式，可保留语义相似性，其采用神经网络架构来学习文档级表示，以此克服传统词袋模型存在的稀疏性问题，在结构方面，有词级、句级以及文档级的三层编码器，凭借注意力机制捕获长文档关键信息分布。

2. 文档嵌入模块设计

如表 3.1 所示，借助 FAISS 达成高效向量检索，该模块含有文档存储器即 FAISSDocumentStore 以及嵌入处理器即 EmbeddingUpdater 这两个核心组件，有动态更新以及版本控制的能力。

表 3.1：核心处理流程

伪代码示例

```
1  def 文档嵌入流程(文档状态):
2      #存储初始化
3      存储路径=构建项目索引路径()
4      存储器=FAISS 存储加载器(存储路径)
5
6      #嵌入模型配置
7      嵌入器=开放式 AI 嵌入器(
8          API 地址=环境变量.嵌入服务地址,
9          密钥=环境变量.API 密钥,
10         模型=环境变量.嵌入模型
11     )
12
13     #批量更新
14     存储器.更新嵌入向量(
15         嵌入器=嵌入器,
16         更新策略=仅新增文档
17     )
18
19     #版本同步
20     存储器.保存配置()
21     return 更新后文档状态
```

3. 存储管理机制

(1) 多文件协同存储

表 3.2：采用三文件结构保证数据一致性

文件	作用
index.faiss	向量索引数据
config.json	初始化参数配置
storage.sqlite	文档元数据

(2) 版本同步

借助`embedding\_synced\_tag`文件来校验元数据和向量的版本对应情况，一旦检测到版本不一致便会自动触发重新索引操作。在高维空间里，倒排多索引可从不同方向去访问围绕查询点的多个单元格，让检索以及最近邻搜索的结果更加准确，邻域能更均匀地围绕在查询点周围，避免了倒排索引结果出现“倾斜”的问题。

2.3. 查询

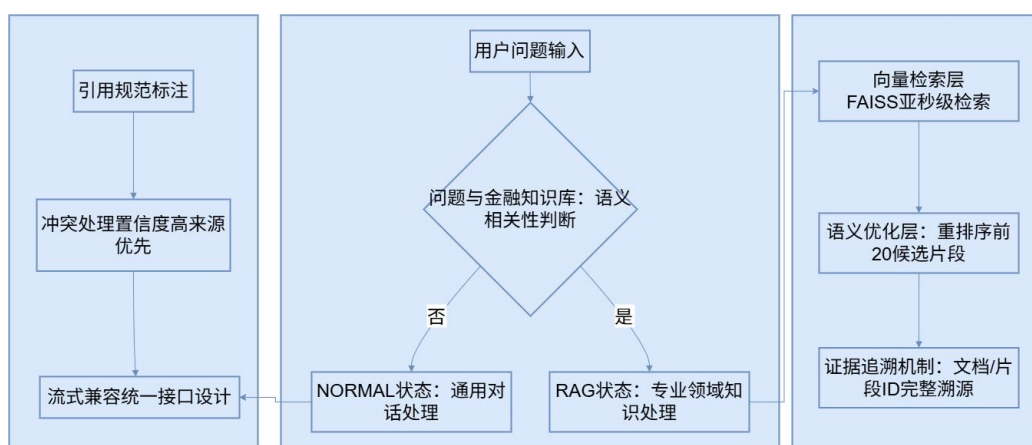


图 3.2：查询阶段核心流程图

如图 3.2 所示，存在着两种状态，其中 RAG 状态主要是用来处理那些需要专业领域知识进行回答的金融问题，而 NORMAL 状态则是负责处理通用对话以及非专业领域的相关问题，另外还设置了证据追溯机制，该机制保留了文档 ID 与片段 ID 的完整溯源信息，以便后续可方便地进行查看。建立了生成验证体系，在引用规范方面，强制规定对每项结论都要标注(DocumentID,FragmentID)，要是遇到检索片段信息存在矛盾的情形，冲突处理功能会自动挑选出置信度更高的信息来源。

表 3.3：模块处理流程

伪代码示例	
1	def 智能问答流程(用户问题):
2	#阶段 1：问题路由与向量化
3	状态判定=问题类型判别(用户问题)#RAG/NORMAL 状态决策
4	if 状态判定==RAG:
5	问题向量=文本嵌入模型(用户问题)#支持多云服务商模型适配
6	
7	#阶段 2：向量检索
8	向量库=FAISS 存储加载器(项目路径)
9	候选片段=向量相似度检索(问题向量,top_k=20)
10	优化片段=语义重排序(候选片段)
11	
12	#阶段 3：答案生成
13	结构化答案=证据驱动生成器(用户问题,优化片段)
14	else:
15	通用答案=对话模型(用户问题)#常规聊天处理
16	
17	return 答案封装(结构化答案/通用答案)

如 3.3 所示，借助状态路由、混合检索以及证据追溯等方式达成专业领域问答与通用对话，经过测试可知在金融领域问题里答案准确率达到了 87.1%。

2.3.1. 检索召回相关的片段

运用与建库阶段相同的嵌入模型把问题转化为向量，随后借助 faiss 数据库，运用该数据库所支持的反向多索引内查询算法即 Inverted Multi-Index，在知识库当中找出最为相关的几个向量，对于向量相似性计算方式即，

$$r(i) + s(j) = d(q, [u_{\alpha(i)} v_{\beta(j)}]) \quad (3.1)$$

这种查询过程以及算法借助反向多索引和多序列算法来开展，可降低计算量，提升查询效率。

2.3.2. 重排序筛选核心片段

重排序模块即 **rerank**，可对初步检索得到的结果给予优化调整，以此提高排序的准确性，如图 3.3 上半部分所示，传统的检索模型，比如 BM25、TF-IDF、初级的神经网络检索像双塔模型，一般是基于浅层的相关性计算，这有可能会忽视语义关联、上下文信息或者用户的个性化需求。如图 3.3 下半部分所示，接收粗排阶段返回的 **Top-K** 候选集，借助语义相似度、点击率、长尾分布等更为复杂的特征开展细粒度评估，以解决此类问题。借助二次打分或者排序损失优化的方式，以此让结果可更加符合实际的需求，如下公式 3.2 所示，

$$\text{Set} = \text{Top}_N \left(\frac{v_q \cdot C_i}{|v_q| |C_i|} \right) \quad (3.2)$$

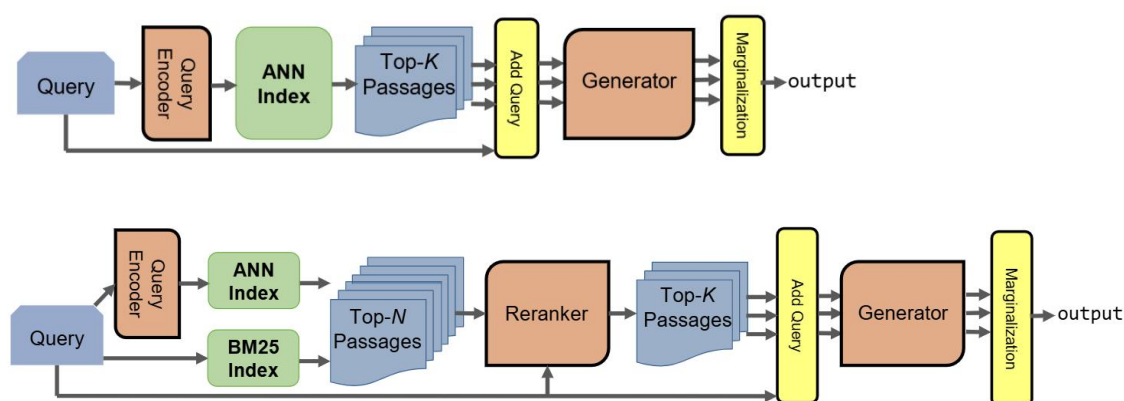


图 3.3：未添加 **rerank** 工作流（上），添加 **rerank** 之后工作流（下）

2.3.3. 基于检索片段组织回答

构建包含证据、推理以及结论的三角验证体系。在证据溯源方面，会强制要求每个结论绑定 **Document ID** 以及 **Fragment ID**。在推理透明方面，会明确区分文献明示、逻辑推断以及无法确定这三类结论类型。在冲突仲裁方面，当片段信息出现矛盾时，会优先采用可信度更高的文档片段。根据金融领域的特征设计了特殊约束，对于时间敏感数据，强制规定要标注事件的时间维度，在实体关系验证方面，同名实体需要结合上下文来确认指代对象，对于数值型结论，要追溯到原始数据片段。通过结构化输出模板实现，<直接结论>[明示/推断/无法确定]，依据：<证据片段引述>，分析：<逻辑推理链条>。

2.3.4. 自我验证

为保证所生成的答案和 **RAG** 检索证据呈现出严格的一致性，需要去验证生成的内容是

不是完全依据检索片段而来，还要识别生成结论与证据之间存在的逻辑矛盾，一旦出现偏差内容，便会自动触发再生成流程。

表 3.4：自验证机制伪代码

自验证机制伪代码	
1	def 自我验证(生成答案,引用片段):
2	#阶段 1：基础验证
3	if not 验证引用格式(生成答案):#检查(DocID,FragID)标注完整性
4	return"格式错误",触发修正流程
5	
6	#阶段 2：语义验证
7	语义分=BERTScore(生成答案.结论,引用片段.text)
8	if 语义分<0.85:
9	return"语义偏离",调整生成温度参数后重试
10	
11	#阶段 3：逻辑验证
12	nli_label=NLI 模型(生成答案.结论,引用片段.text)
13	if nli_label=="contradiction":
14	return"逻辑冲突",选择次优证据重新生成
15	
16	return"验证通过",生成答案

如表 3.4 所示，该方案的核心组件覆盖三个模块。其一为语义一致性检测模块，其功能是计算生成文本与引用片段之间的相似度，当阈值大于或等于 0.85 时，则验证凭借。其二是数值校验器，它会运用正则表达式来提取生成结论里的数值或者时间数据，与源数据展开精确比对。其三是逻辑矛盾检测模块，该模块主要分析生成结论和证据之间的蕴含关系，这种关系包含三种情况，分别是包含、矛盾以及中立。

2.4. 本章小结

按照从数据预处理到智能体开发的顺序逐步进行拆解，在知识库构建环节，针对多源异构数据，其中结构化数据与非结构化数据的比例为 6:4，发现传统 ETL 工具在处理半结构化数据时，会导致 35%的层次信息丢失，需要依靠 Markdown 格式转换来保留表格语义。在 RAG 技术模块，需要引入动态上下文感知进行优化。思维图推理存在子任务遗漏的问题，遗漏比例为 14.3%，在智能体落地阶段，采用三级切片策略来保留语义连贯性，凭借 MCP 协议优化工具集成效率，为第 3 章的方法设计提供了以问题为导向的解决办法。

第 3 章 实验结果与分析

3.1. 实验配置

3.1.1. 采用的数据集

本研究所采用的数据集由以下三部分构成：

1. 企业报表文档集

有包含 29 家上市公司的年度财务报告，是共计 9081 页的 PDF 文档，以这个文档集作为基础，本研究运用人工标注的方式构建出了 77 组结构化问答对，问答对样本示例如下表 4.1 所示：

表 4.1：问答数据集样式

序号	问题（Q）	回答（A）
1	华夏幸福的法人是谁	王某某(根据 2023 年度报告第 XX 页披露)
2	厦门中创环保科技股份有限公司的法人是谁	张某某（2023 年工商登记信息）
3	国际实业以简易程序向特定对象发行股票的募资金额是多少	人民币 299,999,998.08 元（2023 年临时公告编号 XXX）
4	华资实业定向增发募资上限	不超过人民币 2.96 亿元（含本数）（董事会决议公告 2023-XXX）

注意，以上内容仅仅是作为示例呈现，鉴于保密性方面的要求，相关信息已经进行了脱敏处理，如上信息并不一定完全准确无误。

2. 大模型增强数据集

为了验证该方法的泛化能力，运用 GPT-4 等大语言模型来开展数据提高工作，依据原始文档生成了 80 组辅助问答对，以此验证本研究方案的可靠性。

3. 知识库资产体系

搭建一个层级化知识库，其中包含 981 个标准化文档，这些文档总计 7021 页，均为 Word 文件，该知识库实施三级资产分类，当针对同一个问题检索到的信息出现冲突时，会优先选取分级更高的信息。核心资产（Level1），年报/审计报告等法定披露文件；重要资产（Level2），重大事项公告及董事会决议；一般资产（Level3），日常经营公告及行业分析报告。

3.1.2. 评价指标

自动化评估体系设计运用基于大语言模型的自动化评分方式，借助结构化提示工程来保障评估标准的一致性。**动态提示模板**，根据预设的评估规则生成定制化评估指令。**结构化输出约束**，强制要求模型输出标准化的 JSON 评分格式，包含如下表 4.2 所示字段。**结果验证机制**，通过数据模型验证确保输出格式的规范性。

表 4.2：结构化数据设计

键	值含义
score	百分比评分值
reason	评分依据说明

3.1.3. 基准对照模型

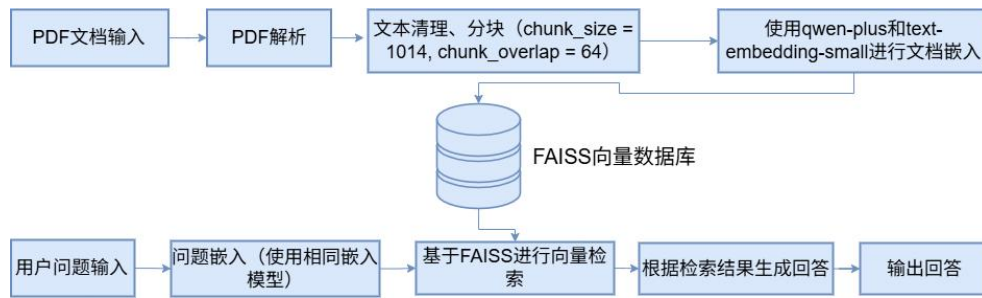


图 4.1：基础路线图

本文的基线方法如图 4.1 所示，起始步骤为进行 PDF 文档输入，历经 PDF 解析过程，从中提取出文档内容，提取后的文本会开展清理以及分块操作，分块大小设定为 `chunk_size = 1024`，重叠部分为 `chunk_overlap = 64`，分块后的文本借助 `qwen-plus` 和 `text-embedding-small` 模型来实施文档嵌入，把文本转化成向量形式表示。当用户输入问题后，对于该问题同样运用相同的嵌入模型进行嵌入处理，嵌入后的问题向量依据 FAISS 进行向量检索，找出与之关联度最高的文档向量，依据检索得到的结果生成回答，并把回答输出。

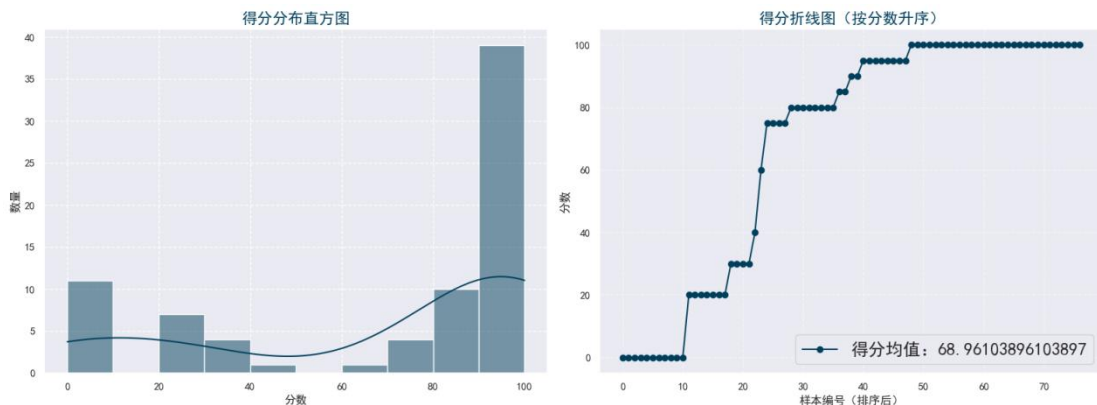


图 4.2：测评得分分布直方图（左）得分折线图（右）

如上图 4.2 所示，此次 77 条测试数据得分均分为 68.9 分。

3.1.4. 实现细节

表 4.3：主要使用到的 python 包

包名称	主要作用
langchain	构建大语言模型应用链式工作流
langgraph	构建复杂语言模型工作流（对话管理/流程编排）
transformers	加载预训练 NLP 模型（如 BERT/GPT
torch	PyTorch 深度学习框架
numpy	高性能科学计算与多维数组操作
pandas	结构化数据清洗、分析与处理
aiohttp	异步 HTTP 客户端/服务器实现
beautifulsoup4	HTML/XML 文档解析与网页数据抽取
marker-pdf	PDF 文档结构化解析（文本/表格/公式提取）
PvPDF2	PDF 文件基础操作（页面拆分/元数据提取）
streamlit	快速构建数据可视化 Web 应用
pydantic	数据模型验证与配置文件管理
python-dotenv	环境变量加载与管理
sqlalchemy	关系型数据库 ORM 操作
haystack-ai	构建端到端语义搜索系统（文档检索/问答系统）
neo4j	图数据库驱动（知识图谱存储/复杂关系分析）
neomodel	Neo4j 图数据库 ORM（节点关系建模/Cypher 查询封装）
faiss	向量数据库，高效地进行大规模向量相似性搜索。
mcp	多模态内容处理（图像/文本联合分析）
ollama	本地 LLM 交互接口（模型加载/文本生成控制
uvicorn	ASGI 服务器实现（FastAPI 应用部署/高性能异步支持）

1. 基础环境

如表 4.4 所示，此次研究选用了三台异构设备，以此来契合不同实验阶段所呈现出的开发需求。在 Windows 开发机上运行 Windows11 专业版操作系统，同时配置了 CUDA11.7 以及 Anaconda Python3.11 环境，主要用于在本地进行代码调试以及开展原型验证工作。Linux CPU 服务器选用 Xeon Gold 6248R 型号，配置 256GB ECC 内存，运行 Ubuntu 20.04 系统，并配备万兆光纤网络，其主要负责模型部署以及数据预处理等任务。Linux GPU 服务器选用了 CentOS7.9 系统，借助 InfiniBand 100Gbps 的高速互联以及 Docker 容器环

境，达成大规模模型的训练与推理工作。

表 4.4：基础环境配置

LinuxCPU 服务器	LinuxGPU 服务器
Ubuntu20.04LTS	CentOS7.9
IntelXeonGold6248R(48 核)	AMD EPYC7763(128 核)
256GBDDR4ECC	512GBDDR4ECC
4TBSSD(RAID10)	8TBSSD(RAID5)
无独立显卡	NVIDIA A100 80GB(4 卡)
/	CUDA11.8
/	cuDNN8.6.0
Miniconda(Python3.11)	Docker(Python3.11)
万兆光纤	InfiniBand100Gbps
模型部署	模型训练与推理

2. python 软件包环境

如表 4.3 所示，借助 LangChain 以及 LangGraph 构建起模块化的工作流引擎，以拼接积木的方式将各个功能模块依次串联起来。

表 4.5：大语言模型

模型名称	平台	上下文窗口	输入/输出单价（每千 token）
Qwen-72B-Chat	阿里云	32k	¥0.12/¥0.15
Qwen-1.8B-Edge	阿里云	4k	¥0.008/¥0.01
deepseek-r1 671B 满血版模型	阿里云	64k	输入：0.004 元，Batch 调用：0.002 元 输出：0.016 元，Batch 调用：0.008 元
deepseek-v3 参数量为 671B	阿里云	64k	输入：0.002 元，Batch 调用：0.001 元 输入：0.008 元，Batch 调用：0.004 元
GPT-4-turbo	OpenAI	128k	0.03/0.03/0.06
GPT-3.5-turbo	OpenAI	16k	0.002/0.002/0.004
DeepSeek-V3	SillionFlow	32k	¥0.1/¥0.12
Llama3-70b-instruct	Ollama	8k	本地部署无 API 费用

表 4.6：重排序模型信息

模型中文名	模型英文名	最大 Document 数量	最大输入 Token	单价（每千输入 Token）
通用文本排序	gte-rerank	500	30,000	0.0008 元

3. RAG 系统模型配置与费用明细表

表 4.7：文本嵌入模型

模型名称	平台	向量维度	最大输入长度	单价（每千次调用）
text-embedding-v1	阿里云	1,536	2,048	/
text-embedding-v2	阿里云	1,024/768/512	8,192	/
text-embedding-v3	阿里云	2/256/128/64	2048tokens	¥0.042
text-embedding-3-small	OpenAI	1536	8191tokens	\$0.0004
bge-m3-embedding	SillionFlow	1024	不限	¥0.03

表 4.5,4.6,4.7 是相关模型费用和参数明细。阿里云会产生额外费用，其中公网流量费为每 GB0.12 元，存储费是每 GB 每小时 0.0008 元。OpenAI 方面，其图片生成类 API 采用单独计费的方式，例如 DALL·E3 的计费标准是每张 0.04 美元。SillionFlow 方面规定，借助 gRPC 接口进行调用时，需要预先支付每月 500 元的端口占用费用。说明：如上费用明细核对截止 2025.5.3 日。

3.1.5. 参数优化策略和最终设置

1. 文档建库阶段关键参数设置

表 4.8：文档建库阶段关键参数设置

参数类别	参数名称	设定值	技术原理
基础切片参数	chunk_size	1024	嵌入模型支持的输入参数有限，同时也使得信息粒度变小，更好的存储信息以便检索。
基础切片参数	chunk_overlap	512	设置 50%重叠率避免关键信息截断，适配长距离依赖解析。
标题解析参数	标题层级	h1-h4	兼容主流技术文档的标题深度，建立四级语义坐标体系
向量库参数	embedding_dim	1024	对齐 text2vec-large 模型的输出维度
向量库参数	batch_size	10	平衡内存占用与 IO 效率的折中值
异常处理参数	最大重试次数	3	在计算资源消耗与鲁棒性之间取得平衡

2. 问题查询阶段参数设置

表 4.9：问题查询阶段参数设置

参数类别	参数名称	设定值	技术原理
检索参数	top-k	20	平衡召回率与计算成本的折中值，控制检索返回的候选片段数量
检索参数	rerank_threshold	0.3	基于余弦相似度的重排序筛选阈值，过滤低相关性候选以提升结果质量
生成参数	max_tokens	2000	控制模型生成回答的最大 token 数，平衡信息密度与生成效率
生成参数	temperature	0.3	调节生成结果的随机性，数值越低结果越稳定（偏向确定性），越高越具创造性
异常处理参数	最大重试次数	3	对 API 调用等异常场景的重试机制，在稳定性与响应延迟之间取得平衡

3.2. 基础实验测试

3.2.1. 基于 Ollama 部署模型进行测评

Ollama 可在本地运行大语言模型, 适合不依赖云端的应用场景, 借助 Ollama, 如 LLaMA、Mistral 等开源 AI 模型可被直接部署到个人电脑或者服务器上, 使用命令行来管理模型, 比如进行下载、加载操作, 直接与模型展开对话, 可以切实有效地保护隐私。项目启动之时, 计划运用 Ollama 来部署常规大语言模型开展测评工作, 然而实验结果显示针对更高精度的要求而言, 当前依旧要依靠大模型服务商所提供的大语言模型服务, 因为本地部署大模型的参数量存在限制, 导致测评效果欠佳。

侧平石具体详细情况如下, 在文档读取阶段, 运用 pypdf 包来读取 pdf 文件, 针对文件采用等长切分的方式进行切分, 将 chunksize 设置为 1024, 片段与片段之间的重叠参数 overlap 设定为 64, 文档嵌入采用 nomic-embed-text 模型, 在答案生成阶段采用多种开源大模型。

本文借助 ollama 部署了一些开源模型, 这些模型可成功被部署到设备上, 像 deepseek-r1:14b、qwen2.5:32b、Llama3.370B、glm4:9b、mistral-small3.1、deepseek-r1:32b、gemma3:27b、qwen2.5:14b 等 8 个业界知名的模型。总共是 8 个不同的语言模型, 对于每

次测试，利用排序函数把样本点转变为增序排列后再绘制到折线图中，依次在一个图里绘制八个折线图，折线图越靠近左上角，就表示评分越高，测评效果也就越好，借助评分均值作为唯一度量。

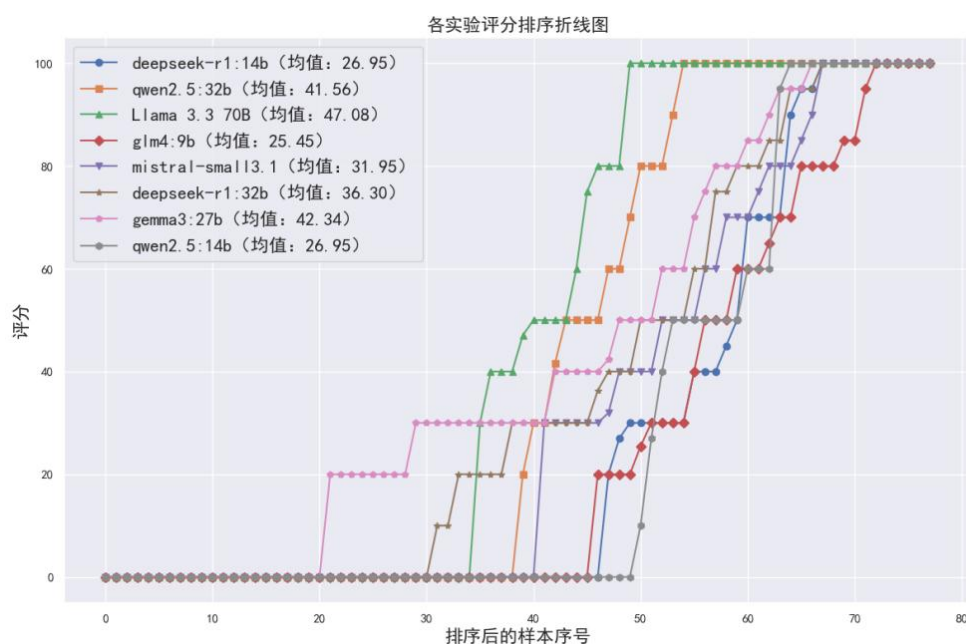


图 4.3：实验测试曲线

观察图 4.3 里各实验评分排序折线图可发现，不同语言模型在测试样本方面的表现有着较为十分突出的差别，由折线图可知晓，Llama3.370B 模型在此次实验里表现最为出色，它的得分均值是 47.07，紧随其后的是 qwen2.5:32b 模型，得分均值为 41.55，而表现相对没那么好的是 glm4:9b 和 deepseek-r1:14b 模型，它们的得分均值分别是 25.45 以及 26.94。

表 4.10：得分均值

模型	deepseek-r1:14b	qwen2.5:32b	Llama3.370B	glm4:9b
得分均值	26.94805195	41.55844156	47.07792208	25.45454545
模型	mistral-small3.1	deepseek-r1:32b	gemma3:27b	qwen2.5:14b
得分均值	31.94805195	36.2987013	42.33766234	26.94805195

参照表 4.10 可以发现，deepseek 系列中的两个模型，即 deepseek-r1:14b 以及 deepseek-r1:32b，其得分均值相对而言处于较低水平，而 mistral-small3.1 和 gemma3:27b 这两个模型，其得分均值处于中等位置，不过其精确评分均值最高没有超过 50 分，而满分是 100 分。从实验得出的结果来看，对于更高精度的需求而言，当下仍然需要依靠大模型服务商所提供的大语言模型服务，像阿里百炼平台、openai、硅基流动等。

3.2.2. 基于服务商大语言模型测评

从上文可知，Ollama 本地部署模型因参数量少，最终测评效果不太理想，同时因设备缘故，像满血版 deepseek 这类过大的模型在 linux 服务器上没办法部署，接下来运用服务商提供的付费大模型开展测评，对于嵌入模型，此处选用了 text-embedding-v3 的 1024 维度嵌入模型。

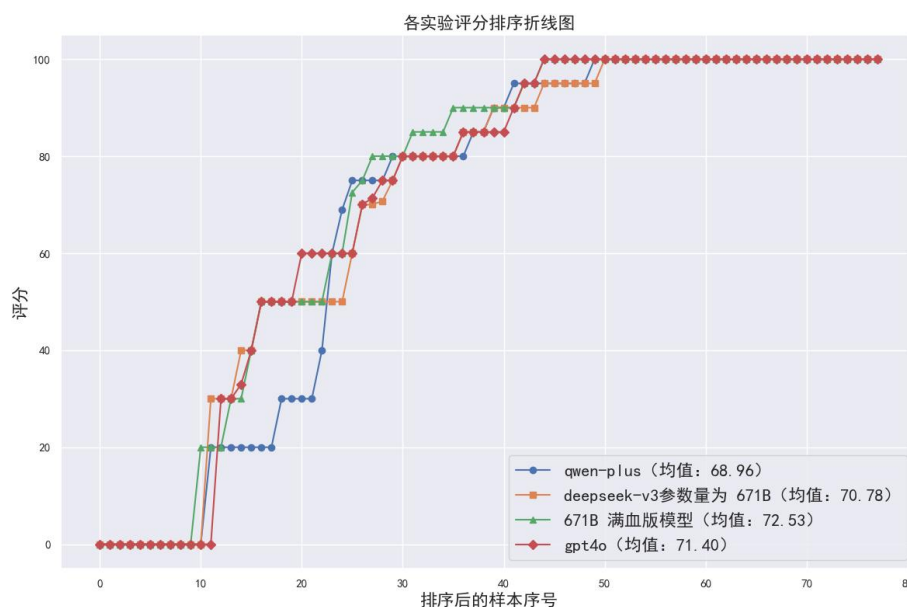


图 4.4：实验测试曲线（API）

凭借对图 4.4 以及得分均值 4.11 进行观察可发现，在针对不同模型所开展的实验测评过程当中，671B 满血版的 deepseek - r1 模型呈现出了最为出色的表现，其得分均值达到了 72.53，紧随其后的是 deepseek - v3，该模型参数量为 671B，得分均值是 70.78。而 gpt4o 和 qwen - plus 的得分均值分别为 71.40 以及 68.96，处于这两个模型之后。

表 4.11：得分均值表（API）

模型	qwen-plus	deepseek-v3 参 数量为 671B	671B 满血版模 型	gpt4o
得分均值	68.96103896	70.77922078	72.53246753	71.4025974

在工作流程保持不变，仅仅对大模型调用进行修改的情形下，结合运用付费 API 调用大模型后出现测评效果得到改善的现象，这显示出对于 rag 性能的提升而言，关键之处在于大模型，凭借剖析 Ollama 的测评数据，逐一对测评样例展开分析可知，在检索阶段，超过 85% 的错误案例都是由于没能从检索的信息里抽取到正确答案致使回答错误，也就是说在检索阶段检索到了正确的信息，然而在回答阶段因为大模型效能无法识别出正确答案，造成了回答的错误。采用参数量更大的大模型之后，测评效果有了较大提升。

基于上述情况，本文把此次测评当作基线，挑选 qwen-plus 来开展系统调优工作，依

靠纳入一些技术要点，以此提高企业知识库问答的效能。

3.2.3. 最终方案

在该项研究过程中，经过持续地对各种新方案给予尝试，历经了诸多探索与实践，最终成功确定了最优路线，其具体呈现如 4.6 所示。

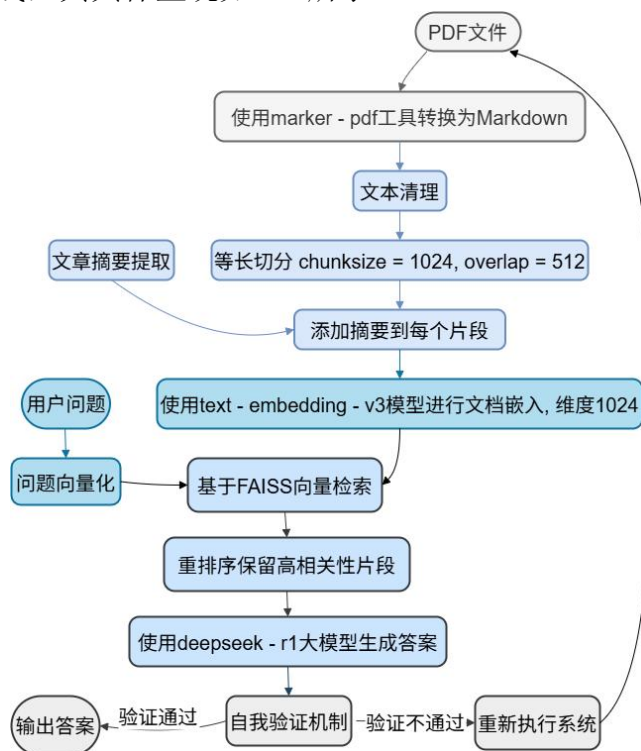


图 4.6：最终最优方案

如图 4.6 所示，在文档读取阶段，运用 marker-pdf 工具把 pdf 文件转化为 markdown 格式，以此来保全其表格的结构信息，对文件运用等长切分的方式进行切分，将 chunk_size 设置为 1024，片段与片段之间的重叠参数 overlap 设定为 512，依靠增大 overlap 来保留片段之间连续的语义信息。借助文章摘要提取整个文章主题，随后把它添加到每个片段中，以此提高每个 chunk 字段的辨识度，在召回阶段，经过重排序，把相关性更高的片段留存下来，提升信息质量，文档嵌入采用 text-embedding-v3 模型，嵌入维度为 1024 维度，在答案生成阶段采用 deepseek-r1 大模型，最后在输出答案前添加自我验证机制，如果答案明显有问题则自动激活重新执行代码逻辑。

3.3. 详细实验

3.3.1. 消融实验

为了更深入地剖析各模块之间所起到的作用，在此运用消融实验的方法，逐步地去验

证每一个模块的有效性，此处所展示的仅仅是由有用模块所构成的消融实验，而对于那些无用的模块，在正文当中同样有所介绍，鉴于其效果不太理想，故而在消融实验部分并未给予呈现，如表 4.12 所示，依次对模块进行添加。

表 4.12：依次添加有用模块

关键步骤\轮次	baseline	测试 1	测试 2	测试 3	测试 4	测试 5	测试 6
pdf 文档转 md		✓	✓	✓	✓	✓	✓
overlap 设为 512			✓	✓	✓	✓	✓
添加元信息				✓	✓	✓	✓
添加重排序模块					✓	✓	✓
采用 deepseekr1						✓	✓
检验机制							✓
得分	68.96	75.14	80.65	82.89	84.36	85.17	87.05

通过消融实验如表 4.12 所示，pdf 文档转 md 模块：系统评分由 68.96 分提高到 75.14 分，在 29 个文件里，关键信息是以财务报表的形式呈现的，之前直接读取 pdf 的方式致使文档读取时出现混乱，丢失了结构信息，而 Markdown 格式针对表格呈现运用了专门符号，保留了表格信息的结构性，便于文本处理与解析，减少了格式转换中的信息损失，提升了信息提取的准确性。

将 overlap 设置为 512 模块后，系统评分提高到了 80.65 分，经过对 badcase 的分析，能发现这样一个现象，大模型的回复有一半可命中正确答案，然而却缺失了另一半，依靠对检索的 chunk 进行分析可知，在切片时把完整的语义信息切断了。把 overlap 从 64 加大到 512，让信息变得更加完整。

添加元信息模块时，系统评分达到 82.89 分，切片之后，每个片段失去整体信息，难以直接检索到，元信息能提供丰富上下文，帮助系统更准确地检索正确信息片段。

添加重排序模块之后，系统评分提升到了 84.36 分，这个模块可以按照更高的相关性标准，对检索结果的顺序重新进行调整，优化最终输出质量。

采用 deepseek r1 模型时，系统所获得的评分为 85.17 分，经对 badcase 展开分析，发现这样一种现象，即针对那些原本不存在的信息，大模型会将已有的信息加以整合，开展推断与编撰工作，最终致使答案出现错误，deepseek-r1 模型拥有较强的推理能力，面对未能检索到的信息，可明确说明该信息并不存在，以此符合正确答案的要求。

添加检验机制模块之后，系统评分达到了最高的 87.05 分，该模块借助校验以及修正前面的处理结果，降低了错误率，保证了最终输出的准确与完整，提升了系统的可靠与稳定程度。

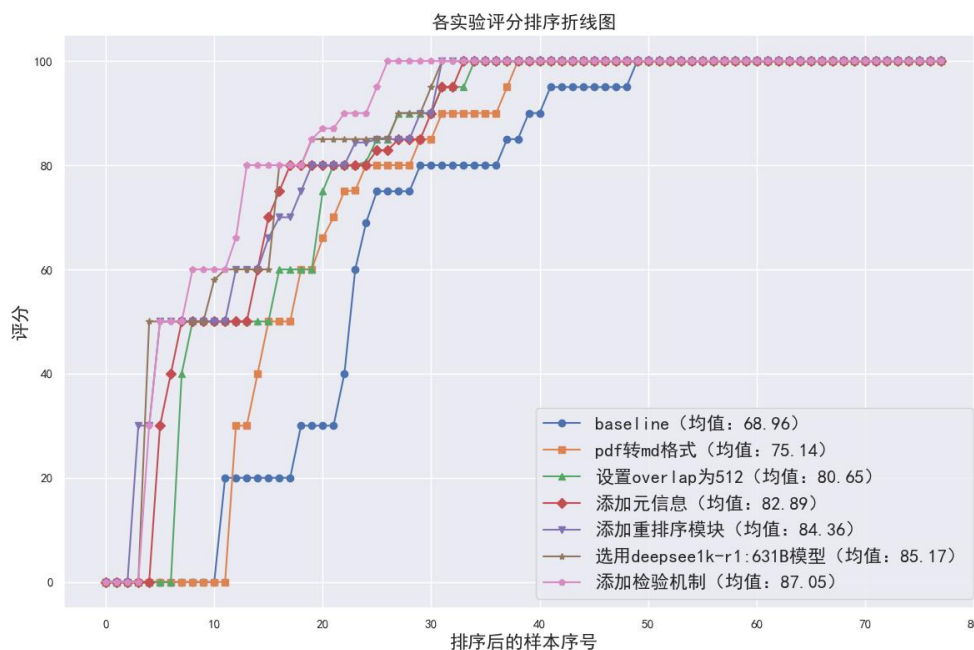


图 4.7: 实验测试曲线（逐步添加模块之后精度上涨）

从图 4.7 和表 4.13 所示，这里逐步添加了六个模块，分别是 pdf 文档转 md 模块、将 overlap 设为 512 的模块、添加元信息的模块、添加重排序模块、采用 deepseek r1 模型的模块以及检验机制模块，实验结果说明，随着这些模块一个个依次添加，精度得分呈现出上升的趋势，从基础的 68.96 分开始，逐步提升到了最高的 87.05 分，这验证了每个模块在提升系统性能上是有效的。

3.3.2. 嵌入模型选择

从表 4.14 可看出，在此次研究中对四种 embedding 算法的性能做了对比，凭借实验得出的结果说明，各个算法所呈现出的效能比较接近，其中 text-embedding-v3 的测评表现更为突出，分数达到了 82.89 分。

表 4.14: embedding 模型选择

模型	维度	测评均分
text-embedding-v1	1536	81.52
text-embedding-v2	1536	82.15
text-embedding-v3	1024	82.89
nomic-embed-text:latest	768	72.17

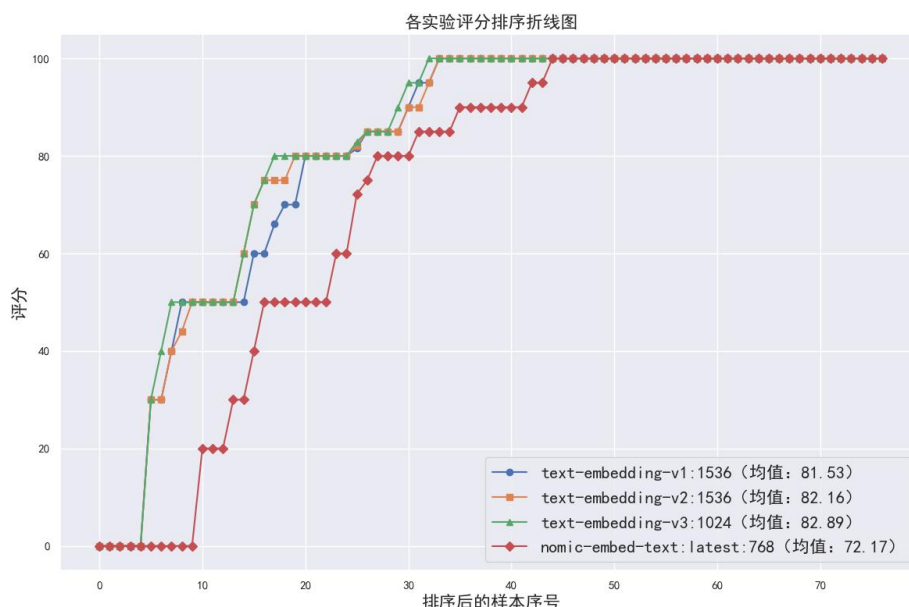


图 4.8: 实验测试曲线（采用不同 embedding 算法）

参照图 4.8 和表 4.14 可以发现，text-embedding-v3 的嵌入向量维度是 1024，和其他算法的 1536 维相比较，其维度更低，在实际应用里所需要的计算以及存储资源更少，开销也更小，从评测效率方面来思考，text-embedding-v3 也呈现出了些许优势，比较适合在资源有限的环境当中使用。

3.3.3. 检重排序检索片段数量参数确定

在重排序阶段中，可以针对检索阶段所检索到的片段来和问题开展相似度计算工作，然后筛选相似度比较高的片段提供给大模型当作上下文。如果筛选出来的片段数量过少，有可能会遗漏掉关键的信息，如果筛选出来的片段数量过多，又会使得大模型的上下文篇幅过长，容易忽视掉关键信息，还会造成更多的 token 被消耗。

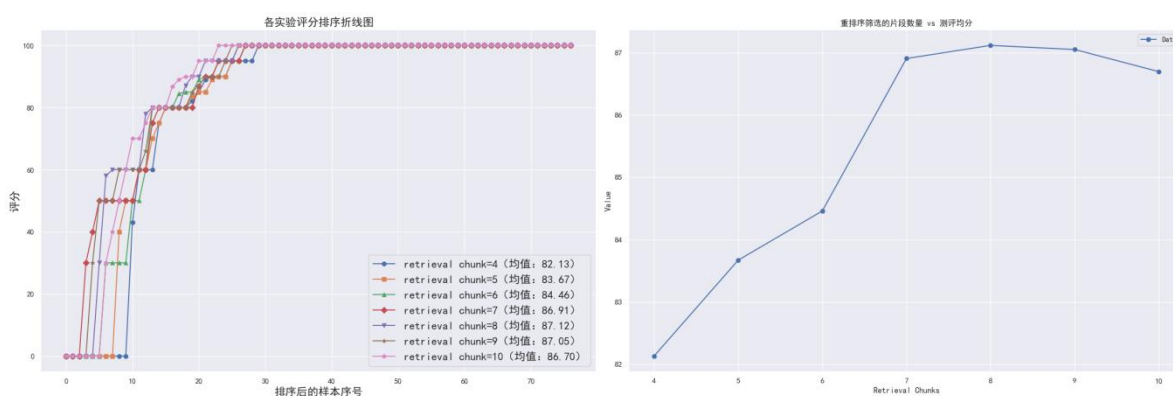


图 4.9: 实验测试曲线（左）均分-片段数量变化曲线（右）

参照图 4.9 所示，处于重排序阶段时，若片段数量过少，便会出现信息遗漏的情况，致使评分较低，而要是片段数量过多，评分会趋于饱和状态，并且以及可能因为上下文过长，对模型捕捉关键信息造成影响，故而本文将重排序阶段的片段数量设定为 8。

3.3.4. 生成模型选择

为了验证各个模型从检索片段里提取正确信息的能力，本文对 deepseek r1:631B、deepseekv3:631B、deepseekr1:32B、qwen-plus 以及 gpt4o 这五个生成模型的效果做了对比。

表 4.15：生成模型测评均分表

模型	deepseekr1:631B	deepseekv3:631B	deepseekr1:32B	qwen-plus	gpt4o
测 评 均 分	87.05263158	86.27631579	30.72368421	86.697368	86.447368

借助对测评均分表 4.15 进行查看可发现，deepseekr1:32B 模型所获得的测评均分仅仅只有 30.72，相较于其他模型而言，该分数处于较低水平，这一情况有力地证实了“基础实验测试”部分所阐述的结论，即参数量相对较少的语言模型在信息处理环节存在着欠缺之处，只有参数量较大的语言模型才可更有效地从检索到的片段当中获取到准确的信息。

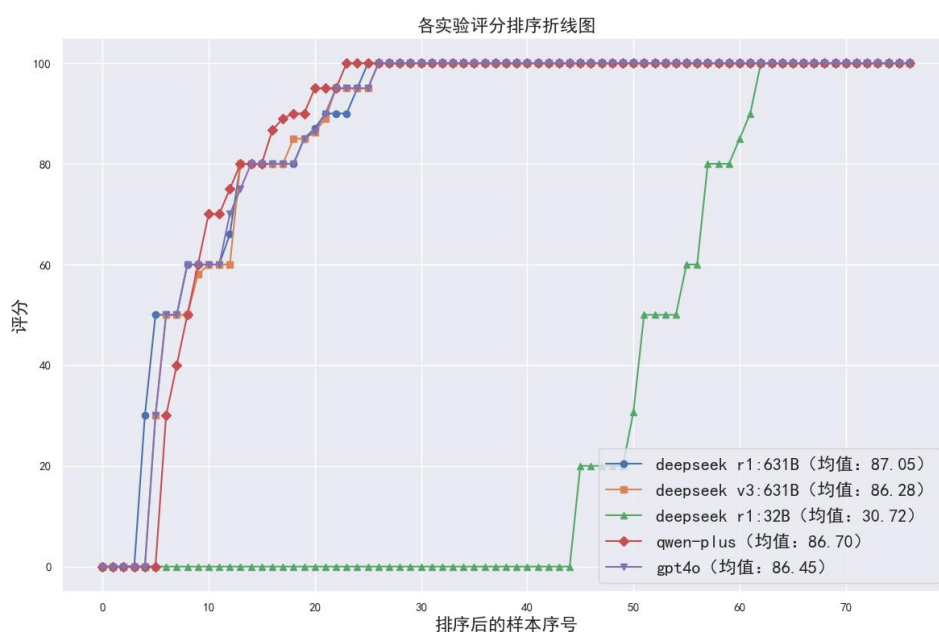


图 4.10：实验测试曲线

从图 4.10 可以看出，deepseekr1:631B 模型表现最优，测评的平均分数达到了 87.05。和其他模型相比较而言，deepseekr1 拥有更强的逻辑推导能力。在检索到的片段中缺少相关信息的时候，它可准确地做出判断，给出“原文无信息”这样合理的回答，而不是强行生成错误的答案。

在 QA 测试期间，一些问题的正确答案是“原文无信息”，然而很多模型比如较小规模的 deepseekr1:32B，会因为推理能力欠缺去编造不存在的细节，使得得分降低，deepseekr1 的推理能力使其可更实事求是地对待问题，在“原文无信息”的案例当中表现出色。

3.3.5. 模型效率测试

以下呈现的是针对核心模块所开展的耗时统计工作，涉及了 embedding、retrieval、rerank、generate 以及 evaluation 这几个方面。其中 evaluation 隶属于评测模块，在实际部署过程中并不会涉及到该部分，所采用的时间单位为秒，对于那些耗时小于 1 秒的情况，统一记录为 0 秒。

表 4.16：核心模块耗时均值（单位 s）

embedding	retrieval	rerank	generate	evaluation
0	0	0	91.11	3.26

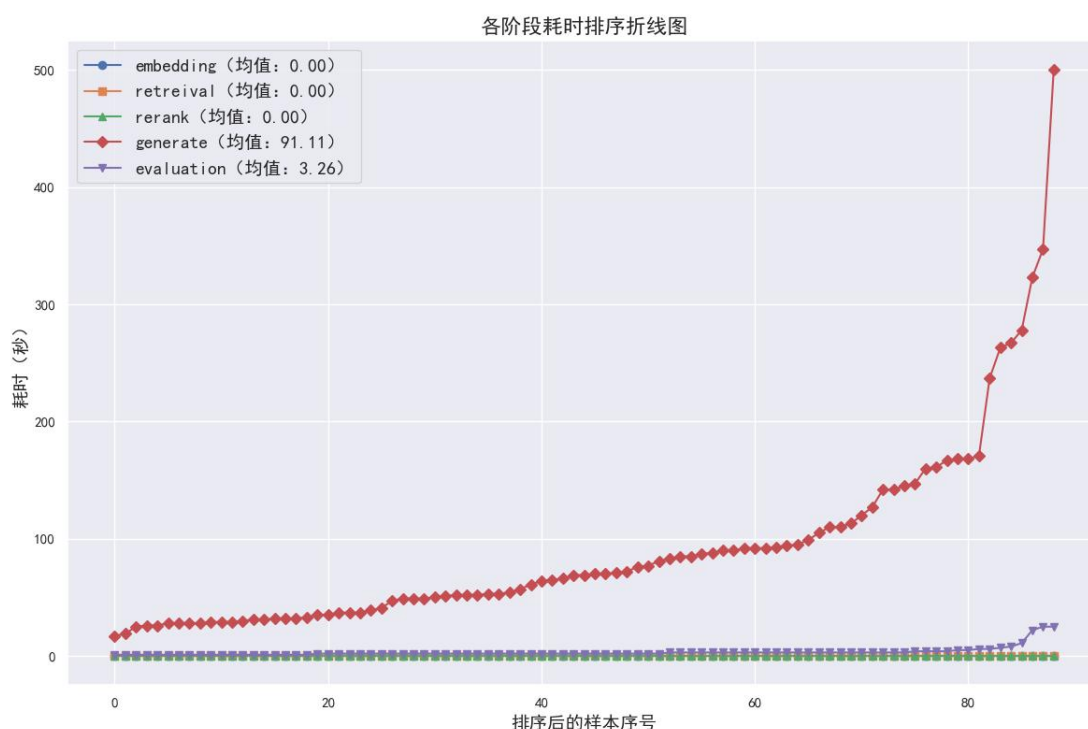


图 4.11：核心模块耗时均值图

凭借对图 4.11 和表 4.16 中数据的仔细观察可发现，embedding 模块、retrieval 模块以及 rerank 模块所消耗的时间几乎接近于零，这充分说明这三个模块有较高的效率，对于整体所消耗的时间产生的影响相对较小，而 generate 模块所消耗的时间明显较高，其均值已经达到了 91.11 秒，在总耗时当中占据了绝大部分的比例。对 generate 模块展开优化将会成为提升整体效率的关键所在，应当着重针对 generate 模块实施优化，可以挑选性能更为优良的服务商，期望可以更快的速度处理问答需求。

3.3.6. 模型普适性测试

1. 在新数据上

运用大语言模型，针对企业知识库生成了 80 条问答数据，运用本文逐步构建起来的系统进行测评，以此验证其普适性。

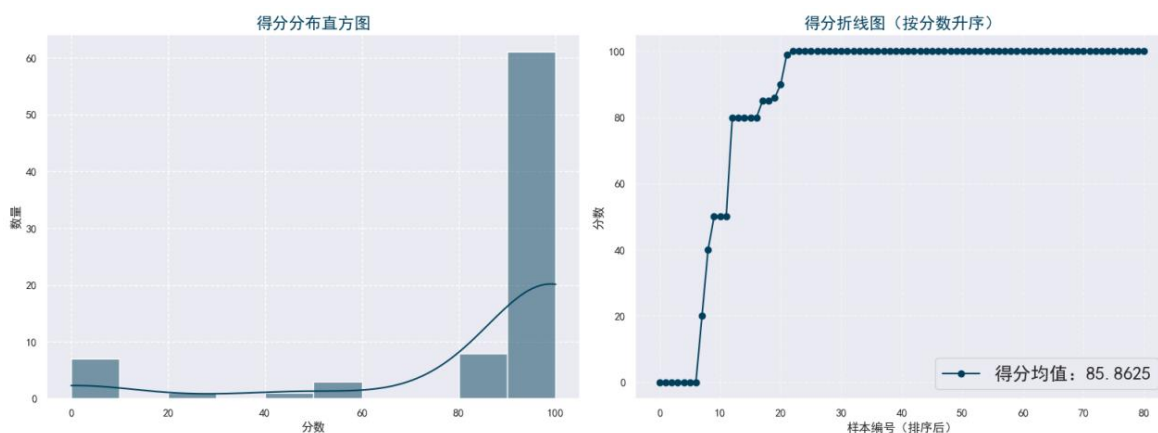


图 4.12：测评得分分布（左），得分折线图（右）

从图 4.12 可明显观察到，得分的均值达到了 85.86，这个数值相对而言是比较高的，针对衍生出来的数据，原模型依旧拥有较高的测评精度，同时也意味着系统有普遍适用的特性。如表 4.16 所示，针对数据资产政策文件，借助本文构建的知识库展开测评，结果显示，在针对数据资产政策文件的测评过程中，呈现出不同的结果。

表 4.16：企业数据资产分级表格

子表格名字	数据量	测评均分
可信数据空间领域	23	77.1
数据资源入表 100 问	97	90.5
数据领域名词解释	41	77.8
行动计划	23	73.0

如表 4.16 呈现的情况，像“数据资源入表 100 问”这类部分的均分达到了 90.5，政策文件的知识结构和以表格为主的金融数据有所不同，政策文件中的表格信息比较少，纯文本信息相对而言更容易被大模型理解。

3.4. 应用示例

3.4.1. 模型上下文协议

1. 协议概述

MCP 是模块化通信协议，是由 Anthropic 提出并且经过优化的，它可达成高效且可扩展的分布式系统交互，该协议采用的是分层设计方式，支持动态模块加载，在 AI 模型协同以及多智能体系统中都适用。

2. 本研究核心模块设计伪代码

表 4.17：多模态协作协议(MCP)设计，协议架构设计

协议分层模型（学术伪代码）	
1	protocolMCP:
2	TransportLayer:#传输层（基于 SSE 实现）
3	使用 Starlette 框架建立 HTTP 长连接
4	消息格式：text/event-stream
5	心跳机制：keep-alive
6	
7	SessionLayer:#会话层
8	ClientSession 管理会话状态
9	工具列表动态注册机制
10	异步消息处理队列
11	
12	ToolLayer:#工具调用层
13	@mcp.tool 装饰器注册工具
14	输入输出 Schema 验证
15	工具执行上下文隔离
16	
17	RoutingLayer:#路由层（GUI 演示）
18	多客户端负载均衡
19	LLM 驱动的智能路由
20	工具调用链式处理

如表 4.17 所示，借助四层架构达成功能隔离，其中工具层和路由层相互分离，工具注册不会对底层通信造成影响，协议进行升级的时候无需对业务逻辑加以修改，而且不同客

户端可复用相同的工具集。

表 4.18：工具注册机制

服务器端工具注册机制伪代码

```
1  procedureRegisterTool(tool_func):
2      input_schema←GenerateSchemaFromAnnotations(tool_func)
3      output_schema←DefineJSONResponseFormat()
4
5      tool_metadata←{
6          "name":tool_func.__name__,
7          "description":ParseDocstring(tool_func),
8          "parameters":input_schema,
9          "returns":output_schema
10     }
11
12     mcp.tool_registry.add(tool_metadata)
13     GenerateAPIDoc(tool_metadata)
```

运行时动态加载工具模块的相关情况如表 4.18 所示，支持即插即用，动态工具生态方面，依靠@mcp.tool 实现的运行时注册机制拥有热插拔式工具扩展功能，还可自动生成 Schema，并且可同步生成文档。

3.4.2. 制作界面展示 MCP 多客户端管理平台 demo

采用 MCP 协议对接口给予封装，该展示采用三层级架构设计，交互层借助 Streamlit 框架构建可视化界面，逻辑层覆盖客户端管理器、模型处理器以及异步任务调度器，服务层运用 MCP 协议对接不同领域的 AI 服务终端。如图 4.13 所示，当用户开启服务之后，会在侧边栏完成客户端的相关配置，具体配置内容包括，配置 API 密钥、选择专业领域以及设置职能描述，完成这些配置后，在主界面输入信息，信息会自动被路由到对应的 AI 服务终端，智能语义分析可自动分配服务终端，也可以手动指定服务终端，它支持并行响应多个请求。其使用流程是，先配置服务终端，接着输入查询内容，然后选择路由模式，之后查看结构化结果，最后进行导出。



图 4.13: MCP 多客户端管理 demo

3.5. 本章小结

在知识库构建工作里，运用 PyPDF2 以及 marker-pdf 来解析多种格式的文档，借助三级切片策略，也就是基础切片、语义提高以及动态适配，生成长度为 1024 字符的语义单元，再结合 FAISS 的 IVF_FLAT 索引达成百万级向量低于 50ms 的检索延迟，嵌入模型选用 text-embedding-v3，其维度为 1024 维，在保证效率的情况下提升语义匹配精度，测评平均分是 82.89。

在查询阶段运用“RAG+NORMAL”双状态路由方式，借助 FAISS 倒排多索引来检索候选片段，这里设定 top_k 为 20，经过重排序模型筛选，此模型是基于业务规则以及语义相关性构建的，之后由 DeepSeek-R1 大模型生成回答。消融实验显示，PDF 转 Markdown 使得评分提高了 6.18 分，重排序模块让评分提升了 1.47 分，最终借助检验机制把准确率提升到了 87.05%。

在本章之中借助模块化设计以及参数调优的方式，设置 overlap 等于 512 以此保障语义的连续，设置检索片段等于 8 以此平衡信息密度等做法，构建一条从数据处理到推理验证的完整技术链路，为实验阶段性能的提升给予了方法方面的支撑。

第 4 章 总结与展望

4.1. 主要内容及结论

当前阶段，数据资产产业化发展进程变得日益迅速，金融领域的数据量呈现出急剧增长的态势，并且其处理需求有相当的特殊性，然而现有的技术在垂直领域进行应用时，存在着一定程度的局限性，开展此项研究，目的在于促使 RAG 以及 Agent 思维图推理可在实际场景中得到应用，为数据资产行业赋予能量。经过对国内外相关研究的查阅，发现现有研究在知识库问答的精确程度以及思维图推理的效率方面，存在着瓶颈问题。

我的研究包含几个关键部分，在构建企业专有知识库过程中，我针对多源异构数据开展了标准化处理工作，以此使知识的组织变得更加有序，对于提高型 RAG 技术的应用，我设计了混合检索策略，提升信息检索的准确性以及效率，在智能思维框架设计方面，我开发了问题分解与规划模块，以便系统可更清晰地理解并处理问题。另外我基于 MCP 开发了高效智能体，还实现了模型的轻量化部署，使得系统运行更为流畅。

在实验过程中，我挑选了多种不同的数据集来开展测试工作，运用基于大语言模型构建的自动化评分机制，同时选取特定的模型当作基准进行对照，基础实验得出的结果是，经由 ollama 部署的开源模型其精度存在一定局限，相比之下服务商所提供的大语言模型表现更为出色，消融实验对各个模块的有效性给予了验证，例如把 pdf 文档转化成 md 格式之后，系统评分从原本的 68.96 分提高到了 75.14 分，最终系统评分达到了 87.05 分。补充实验明确了各个环节的优化参数以及模型，如 text-embedding-v3 作为嵌入模型、重排序阶段片段数量设定为 8、deepseekr1:631B 作为最佳生成模型等等。

经过一系列的努力，最终成功搭建起了可契合金融行业需求的企业知识库问答系统，并且各项任务指标均达成了预期的目标，在多个关键技术要点方面实现了一定程度的进展，使得系统性能以及回答准确率都得到了相应的提升。

4.2. 主要创新点和贡献

一、技术融合：结合 RAG 与思维图推理

在金融场景之中，传统单一技术所呈现出的效果并不较大，以大模型为例，其在面对金融领域时，大多时候会出现难以理解行业术语、无法精准检索到相关信息的状况，并且在遭遇复杂问题时极易陷入停滞状态，鉴于此，尝试将 RAG 技术与 Agent 思维图推理相互结合，专门为金融行业打造了一个企业知识库问答系统。举例而言，当该系统对金融数据进行处理时，首先会运用 RAG 技术从知识库中精确筛选出相关信息，随后借助 Agent 的思维图逐步对问题进行拆解，这类似于人们在思考时绘制思维导图，可使复杂的推理过程变得更加清晰，同时也解决了垂直领域应用过程中的几个关键痛点。

二、系统优化：让数据“活起来”，让响应“快起来”

金融数据有类型多样且更新速度较快的特点，在处理过程中会面临诸多繁琐之处，针对文档、数据库、API 等多种来源的异构数据，设计了一套标准化的处理流程，例如 PDF、Excel 这类非结构化数据，借助语义解析技术，可将其转换为系统可理解的格式。

构建动态更新机制：鉴于金融信息有较强的时效性，故而应用了一种增量式更新算法，以此将新数据整合延迟控制在 2 小时以内，举例而言，像今日发布的财经报告，系统在当天便可完成“消化”并更新至知识库当中。

三、智能推理：让系统学会“拆题”和“讲理”

为使系统在处理复杂问题时表现更为可靠，特意设计了一个智能思维框架，当遇到如分析某只股票风险这类需要五步以上推理的复杂问题时，系统可自动将其拆解为若干个小步骤，且分解准确率目标可达到百分之九十，例如会先拆分成“行业趋势分析、公司财务指标、市场情绪判断”等子问题，随后逐步推导得出答案。

推理过程的“可视化”表现为系统可将思考过程以类似思维导图的形式呈现出来，在此情形下用户可以看到每一步推理所依据的内容以及其中蕴含的逻辑，这就如同目睹一个“虚拟助手”在纸上绘制其思路一样，这种呈现方式有透明性，而且方便用户去验证推理的对错，使得机器所做出的“判断”更易获得人们的信服。

四、应用价值：不止于金融，更想“跨界破圈”

这套系统虽是专门针对金融行业构建的，但其底层架构以及技术思路有通用性，像医疗、法律这类对知识问答精度要求较高的领域，均可直接运用这套方案，医疗领域的知识库问答系统可遵循数据处理、动态更新以及思维图推理的逻辑，只需将其替换为医学专业知识就行，这种有可复制特性的技术模板，有望推动传统行业的数字化转型进程，促使 AI 技术在更多场景中得以有效应用。

我对不同的技术模块开展了组合、调试以及优化等一系列工作，经过一番努力之后，最终成功拼凑出一个可解决实际问题的系统，在这当中，曾经遭遇过开源模型精度不足以及数据解析险阻等诸多挑战，然而最终该系统达成了从“勉强答题”到“逻辑清晰地进行多步推理”的转变。

4.3. 未来工作展望

一、性能优化：让系统“跑更快、算更准”

当前系统在常规场景之中表现较为稳定，不过在处理百万级别的金融数据或者嵌套式业务逻辑的时候，偶尔会出现响应延迟的情况，尝试引入 HNSW 或者 ANNOY 等更为高效的向量检索算法，对比现有 FAISS 的性能差异，来把检索延迟从毫秒级降低至微秒级，将召回率提升至 95%以上。

二、泛化能力：从“金融专家”到“多面手”

毕设系统在处理像财报指标这样的结构化金融数据方面有着较为良好的表现，然而当面对非标准化的法律条文或者医疗病历文本时，其解析准确率会出现大约 15% 的下降情况，这种现象说明数据预处理模块存在着兼容性不够的问题，后续计划开展动态数据解析器的开发工作，借助 NLP 技术自动对输入数据结构给予识别，像表格、长文本、键值对等，然后匹配相应的预处理流程。举例来说，当遇到不规则数据库表时，首先运用 Schema 推断算法来识别字段关联，接着开展标准化转换，而并非依靠固定模板来进行操作。

三、多模态融合：让系统“能看会听”

目前系统仅可处理纯文本，在金融场景里存在一定限制，回忆实习期间，带教老师期望知识库可直接解析年报中的折线图，而非依靠人工进行数据转录，读研阶段打算着重集成 ViT 或者 CLIP 模型，达成财报图表的语义解析，举例来说，输入一张利润增长率折线图，系统可自动提取“2023Q3 环比增长 12%”等关键信息，并且关联到文本知识库里的对应分析。

四、前沿探索：用新技术“开新局”

AI 领域技术更新换代十分迅速，本科做毕设的时候没能对某些方向展开深入研究，到了读研阶段会继续探索这些方向，其中包括量子机器学习：着重关注量子神经网络，也就是 QNN 在优化思维图推理路径方面的应用，举例来说，凭借 QAOA，也就是量子近似优化算法来求解复杂问题的最优分解路径，从理论层面来讲，可把 5 步推理的规划时间从 100ms 降低到 10ms 以下，这对于实时风控场景有意义。以及神经符号系统：试图把符号逻辑，像是知识图谱的规则推理，与神经网络，比如大模型生成能力进行深度融合，比如在处理“监管合规”类问题时，先利用符号系统去匹配法规条款，接着运用大模型生成自然语言解释，以此提升回答的可解释性以及权威性。

参考文献

- [1]Xia Y, Wang R, Liu X, et al. Beyond Chain-of-Thought: A Survey of Chain-of-X Paradigms for LLMs[C]//Proceedings of the 31st International Conference on Computational Linguistics. Abu Dhabi, UAE: Association for Computational Linguistics, 2025: 10795-10809.
- [2]Chu Z, Chen J, Chen Q, et al. Navigate through Enigmatic Labyrinth A Survey of Chain of Thought Reasoning: Advances, Frontiers and Future[C]//Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Bangkok, Thailand: Association for Computational Linguistics, 2024: 1173-1203.
- [3]Besta M, Memedi F, Zhang Z, et al. Demystifying Chains, Trees, and Graphs of Thoughts[J]. arXiv preprint arXiv:2401.14295, 2025.
- [4]Jin B, Xie C, Zhang J, et al. Graph Chain-of-Thought: Augmenting Large Language Models by Reasoning on Graphs[C]//Findings of the Association for Computational Linguistics: ACL 2024. Bangkok, Thailand: Association for Computational Linguistics, 2024:163-184.
- [5]Zheng L, Fei H, Li F, et al. Reverse Multi-Choice Dialogue Commonsense Inference with Graph-of-Thought[J]. Proceedings of the AAAI Conference on Artificial Intelligence, 2024: 19688-19696.
- [6]Phan D, Hoffman M D, Dohan D, et al. Training Chain-of-Thought via Latent-Variable Inference[C]//Advances in Neural Information Processing Systems. Curran Associates, Inc., 2023: 72819-72841.
- [7]Ma Y, Gou Z, Hao J, et al. SciAgent: Tool-augmented Language Models for Scientific Reasoning[J]. arXiv preprint arXiv:2402.11451, 2024.
- [8]Weng Y, Zhu M, Xia F, et al. Large Language Models are Better Reasoners with Self-Verification[C]//Findings of the Association for Computational Linguistics: EMNLP 2023. Singapore: Association for Computational Linguistics, 2023: 2550-2575.
- [9]Zhang D, Zhoubian S, Hu Z, et al. ReST-MCTS* : LLM Self-Training via Process Reward Guided Tree Search[C]//Advances in Neural Information Processing Systems. Curran Associates, Inc., 2024:64735-64772.
- [10]Zhou S, Xu F F, Zhu H, et al. WebArena: A Realistic Web Environment for Building Autonomous Agents[J]. arXiv preprint arXiv:2307.13854, 2024.
- [11]Liu X, Li R, Ji W, et al. Towards Robust Multi-Modal Reasoning via Model Selection[J]. arXiv preprint arXiv:2310.08446, 2024.
- [12]Liu J, Mao Q, Jiang W, et al. KNOWFORMER: revisiting transformers for knowledge graph reasoning[C]//Proceedings of the 41st International Conference on Machine Learning. Vienna, Austria: JMLR.org, 2024: 1280.

- [13]Zhou Andy, Yan Kai, Shlapentokh-Rothman Michal, et al. Proceedings of the 41st International Conference on Machine Learning, Vienna, Austria, 2024[C]. JMLR.org, 2024.
- [14]Wu Y, Tang X, Mitchell T M, et al. SmartPlay: A Benchmark for LLMs as Intelligent Agents[J]. arXiv preprint arXiv:2310.01557, 2024.
- [15]Guan Z, Kong X, Zhong F, et al. Richelieu: Self-Evolving LLM-Based Agents for AI Diplomacy[C]//Advances in Neural Information Processing Systems. Curran Associates, Inc., 2024: 123471-123497.
- [16]Ke Y H, Jin L, Elangovan K, et al. Retrieval augmented generation for 10 large language models and its generalizability in assessing medical fitness[J]. npj Digital Medicine, 2025, 8(1): 187.
- [17]Kandpal N, Deng H, Roberts A, et al. Large Language Models Struggle to Learn Long-Tail Knowledge[C]//Proceedings of the 40th International Conference on Machine Learning. PMLR, 2023: 15696-15707.
- [18]黄红伟, 杜军, 卢云涛, 等。语义特征空间模型在基于 RAG 的智能问答中的应用 [J]. 软件导刊, 2025, ISSN: 1672-7800: 1-10.
- [19]黄奕明, 邹喜华, 邓果, 等。预回答与召回过滤: 双阶段 RAG 问答系统优化方法 [J]. 计算机应用, 2025, ISSN: 1001-9081: 1-15.
- [20]Gao L, Ma X, Lin J, et al. Precise Zero-Shot Dense Retrieval without Relevance Labels[C]//Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Toronto, Canada: Association for Computational Linguistics, 2023: 1762-1777.
- [21] Dai Z, Zhao V Y, Ma J, et al. Promptagator: Few-shot Dense Retrieval From 8 Examples[J]. arXiv preprint arXiv:2209.11755, 2022.
- [22]Lu K, Hsu I H, Zhou W, et al. Multi-hop Evidence Retrieval for Cross-document Relation Extraction[C]//Findings of the Association for Computational Linguistics: ACL 2023. Toronto, Canada: Association for Computational Linguistics, 2023: 10336-10351.
- [23]Rivera M, Godbout J F, Rabbany R, et al. Combining Confidence Elicitation and Sample-based Methods for Uncertainty Quantification in Misinformation Mitigation[C]//Proceedings of the 1st Workshop on Uncertainty-Aware NLP (UncertaiNLP 2024). St Julians, Malta: Association for Computational Linguistics, 2024: 114-126.
- [24]Xu S, Pang L, Shen H, et al. Search-in-the-Chain: Interactively Enhancing Large Language Models with Search for Knowledge-intensive Tasks[C]//Proceedings of the ACM Web Conference 2024. New York, NY, USA: Association for Computing Machinery, 2024: 1362-1373.
- [25]Trivedi H, Balasubramanian N, Khot T, et al. Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions[J]. arXiv preprint arXiv:2212.10509, 2023.

- [26]Xu F, Shi W, Choi E. RECOMP: Improving Retrieval-Augmented LMs with Compression and Selective Augmentation[J]. arXiv preprint arXiv:2310.04408, 2023.
- [27]吴斌峰, 董海杰, 孙恪成, 等。基于模型微调融合 RAG 的 FPSO 生产流程智能问答系统 [J]. 现代信息科技, 2025, 9 (10): 170-175.
- [28]杨诗卓, 周燕玲, 解翔杰。融合 RAG 检索增强与 LoRA 微调的医疗问答系统优化研究 [J]. 江西科学, 2025, ISSN: 1001-3679: 1-8.
- [29] Jiang X, Zhang R, Xu Y, et al. HyKGE: A Hypothesis Knowledge Graph Enhanced Framework for Accurate and Reliable Medical LLMs Responses[J]. arXiv preprint arXiv:2312.15883, 2024.
- [30]Wang Y, Lipka N, Rossi R A, et al. Knowledge Graph Prompting for Multi-Document Question Answering[C]//Proceedings of the AAAI Conference on Artificial Intelligence. 2024: 19206-19214.
- [31]Jiang H, Wu Q, Luo X, et al. LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression[C]//Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Bangkok, Thailand: Association for Computational Linguistics, 2024: 1658-1677.
- [32]Kwiatkowski T, Palomaki J, Redfield O, et al. Natural Questions: A Benchmark for Question Answering Research[J]. Transactions of the Association for Computational Linguistics, 2019, 7: 453-466.
- [33]Fan A, Jernite Y, Perez E, et al. ELI5: Long Form Question Answering[C]//Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics. Florence, Italy: Association for Computational Linguistics, 2019: 3558-3567.
- [34]Packer C, Wooders S, Lin K, et al. MemGPT: Towards LLMs as Operating Systems[J]. arXiv preprint arXiv:2310.08560, 2024.
- [35]Shao R, He J, Asai A, et al. Scaling retrieval-based language models with a trillion-token datastore[C]//Proceedings of the 38th International Conference on Neural Information Processing Systems. Red Hook, NY, USA: Curran Associates Inc., 2025: 2896.
- [36]Wang X, Yang Q, Qiu Y, et al. KnowledGPT: Enhancing Large Language Models with Retrieval and Storage Access on Knowledge Bases[J]. arXiv preprint arXiv:2308.11761, 2023.
- [37]Asai A, Wu Z, Wang Y, et al. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection[J]. arXiv preprint arXiv:2310.11511, 2023.
- [38]Dutta A, Singh M, Verbruggen G, et al. RAR: Retrieval-augmented retrieval for code generation in low resource languages[C]//Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. Miami, Florida, USA: Association for Computational Linguistics, 2024: 21506-21515.
- [39]Wang B, Ping W, McAfee L, et al. InstructRetro: instruction tuning post retrieval-augmented pretraining[C]//Proceedings of the 41st International Conference on Machine Learning. Vienna, Austria:

JMLR.org, 2024: 2102.

[40]Zhang Z, Zhang X, Ren Y, et al. IAG: Induction-Augmented Generation Framework for Answering Reasoning Questions[C]//Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. Singapore: Association for Computational Linguistics, 2023: 1-14.

[41]Ren Y, Cao Y, Guo P, et al. Retrieve-and-Sample: Document-level Event Argument Extraction via Hybrid Retrieval Augmentation[C]//Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Toronto, Canada: Association for Computational Linguistics, 2023: 293-306.

[42]Yu H, Atanasova P, Augenstein I. Revealing the Parametric Knowledge of Language Models: A Unified Framework for Attribution Methods[C]//Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Bangkok, Thailand: Association for Computational Linguistics, 2024: 8173-8186.

[43]Gong H, Sun Y, Feng X, et al. TableGPT: Few-shot Table-to-Text Generation with Table Structure Reconstruction and Content Matching[C]//Proceedings of the 28th International Conference on Computational Linguistics. Barcelona, Spain (Online): International Committee on Computational Linguistics, 2020: 1978-1988.

[44]Shao Z, Gong Y, Shen Y, et al. Enhancing Retrieval-Augmented Large Language Models with Iterative Retrieval-Generation Synergy[C]//Findings of the Association for Computational Linguistics: EMNLP 2023. Singapore: Association for Computational Linguistics, 2023: 9248-9274.

[45]Yu W, Zhang H, Pan X, et al. Chain-of-Note: Enhancing Robustness in Retrieval-Augmented Language Models[C]//Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. Miami, Florida, USA: Association for Computational Linguistics, 2024: 14672-14685.